



생성형 AI 플랫폼 GenOS 기능 교육

제논 | 2026.09



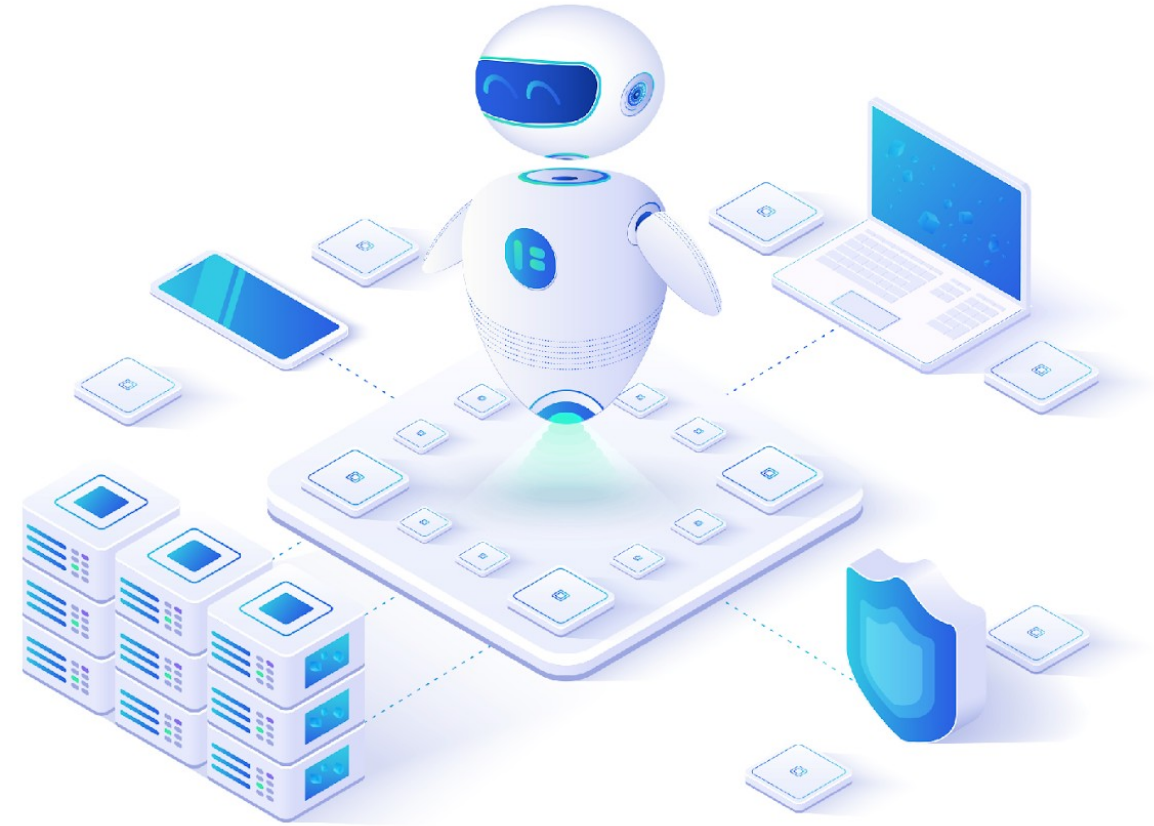
INDEX



- I. 워크플로우와 Flowise
- II. GenOS 리소스 활용
- III. 채팅 애플리케이션 연동
- IV. 에이전트 개발

I 워크플로우와 Flowise

1. 워크플로우
2. Flowise
3. 워크플로우 활용 사례



1. 워크플로우

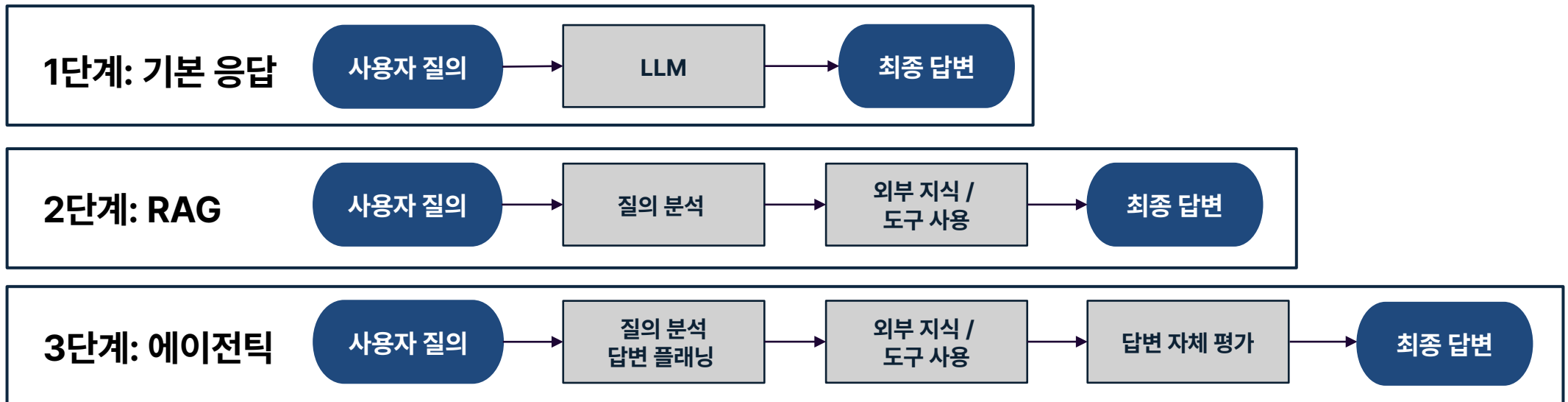
워크플로우 & AI 워크플로우

워크플로우

- 특정 업무 (목표) 에 대한 정보를 처리하는 절차, 작업 흐름

AI 워크플로우

- AI(LLM), 프롬프트, 데이터 소스, 도구들을 논리적으로 연결하여 작업을 수행하는 파이프라인



Flowise 개요

복잡한 LLM 워크플로우를 시각적 노드로 설계하고 배포할 수 있는 오픈소스 노코드/로우코드 플랫폼

Flowise

주요 기능

- **No-Code/Low-Code**
노드를 기반으로 워크플로우를 노코드/로우코드로 구성할 수 있음
- **노드별 모델 설정 및 멀티에이전트 구성**
여러 에이전트가 협업하는 워크플로우 구성 가능
노드별 상이한 LLM 설정 가능
워크플로우 내 다른 에이전트 호출 가능
- **LLM 기반 분기 처리 및 도구 사용**
모델이 사용자 요청에 적합한 도구를 선택 및 실행할 수 있으며 분기 처리도 가능

GenOS 워크플로우 활용 사례

1

사규 및 컴플라이언스 RAG

- 사규와 관련 법령 RAG
- 법령 변경 시 사규에 미칠 영향 검토 가능

2

데이터 분석 에이전트

- 사용자 질의를 분석하여 Text2SQL
- 복수 테이블 Join 및 시각화
- Human In The Loop

3

리서치 에이전트

- 사용자 질의를 분석하여 적절한 외부 검색
- 외부 검색 결과 시각화 필요 시 수행

4

작성 가이드 기반 첨삭 어시스턴트

- 사내 작성 가이드에 기반한 문서 내용 교정
- 첨삭 사유 및 근거 제시
- 파일 업로드 지원

5

제약 리서치 에이전트

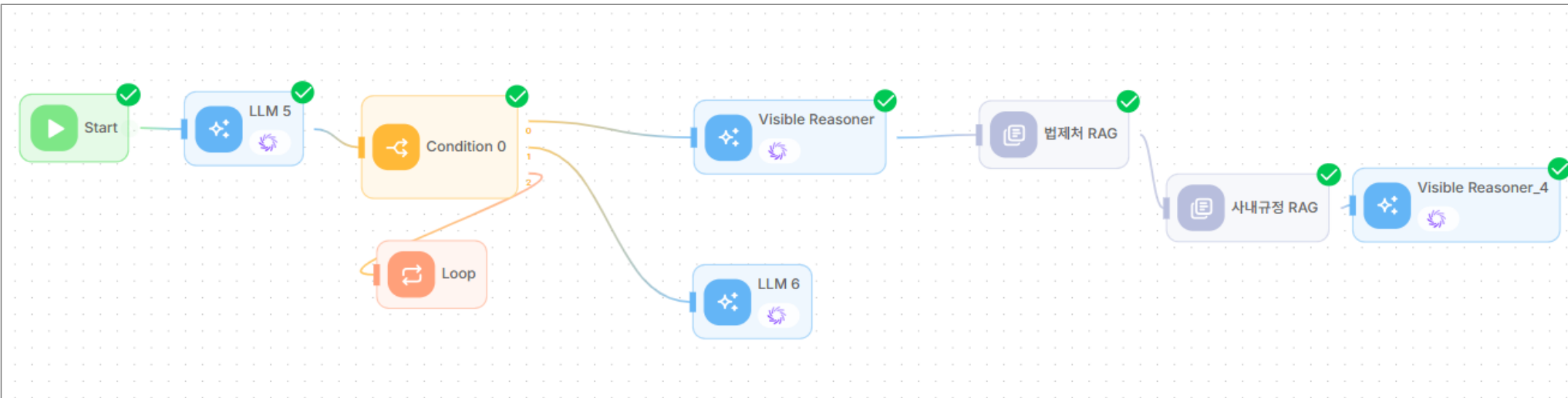
- 리서치 플래너가 질의에 필요한 에이전트만 선택하여 답변 생성 진행
- 외부 MCP 도구 활용

3. 워크플로우 활용 사례

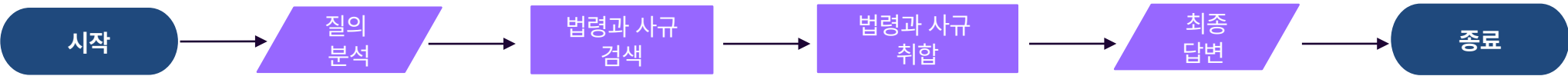
사규 및 컴플라이언스 RAG

사규 및 법령 벡터DB 동시 검색으로 개정 사항 발생 시 관련 규정 간 영향 파악

워크플로우



• 워크플로우 로직



3. 워크플로우 활용 사례

리서치 에이전트

최종 답변을 도출하고자 작업 계획을 세운 뒤 이를 기반으로 체계적으로 리서치 수행

워크플로우

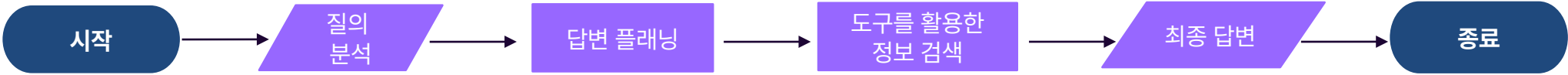
최근 3년간 금융 산업에서 생성형 AI 도입 사례와 그 영향을 연도별로 테이블로 정리하고, 주요 변화 요인을 요약해 드리겠습니다.

연도	생성형 AI 도입 사례 및 활용 내용	주요 영향 및 변화
2022년	- 12월 OpenAI 챗GPT 출시 및 금융권 관심 급증	- 생성형 AI 기술 첫 등장, 금융업계의 디지털 트랜스포메이션 가속화
	- 초기에는 내부 직원 업무 지원, 문서 요약, 보고서 작성 보조에 활용	- 업무효율성과 정확성 향상 가능성 인식 시작
2023년	- KB금융 'KB-GPT' 업무지원 AI뱅크 도입	- 금융기관 내부보고서 작성, 상품 홍보문구 작성 등 업무 효율성 증대
	- 금융권 AI 고객상담센터(AICC)에서 챗봇 고도화 및 고객 서비스 개선	- 고객응대 자동화로 고객만족도 및 신뢰도 증대
	- 미국 JP모건체이스 400여개 업무에 생성형 AI 적용	- 자산관리, 재무예측, 이상거래 탐지 등 업무 효율성 증대
	- 캐피탈원 등 사기 탐지 시스템에 생성형 AI 도입	- 사기거래 탐지 정확도 향상 및 리스크 관리 강화
2024년	- 국내 NH농협, 신한은행 등 대출심사 및 추천서비스에 AI 활용 확대	- AI 금융상품 추천, 기업 대출 심사 등 금융상품 개발 및 고객 맞춤형 서비스 강화
	- 신한라이프 등 보험사 AI 조직 구축 및 DX 추진	- 금융상품 개발 및 고객 맞춤형 서비스 강화
	- JP모건체이스 400개 이상 AI 활용 사례 확대	- 업무 자동화와 고객 맞춤형 자산관리 고도화
	- 금융권 AI 플랫폼 구축 통한 상용·오픈소스 AI 전략적 활용	- AI 도입 범위 확대 및 디지털 금융 채널 변화
	- 금융산업 내 AI 거버넌스 강화, 규제 개선 필요성 부각	- AI 영업 리스크, 정보보호, 윤리 문제 대응 체계 마련
2025년	- 생성형 AI 활용 금융산업 혁신 가속화 전망	- 디지털금융 개인화, 자동화 심화, 금융산업 생태계 변화
	- AI 기반 리스크 관리 및 고객 상담 고도화 지속	- 고객 서비스 품질 향상 및 금융사고 예방 가능

최근 3년간 금융 산업에서 생성형 AI 도입 사례와 그 영향을 연도별로 테이블로 정리하고, 주요 변화 요인을 요약해 드리겠습니다.

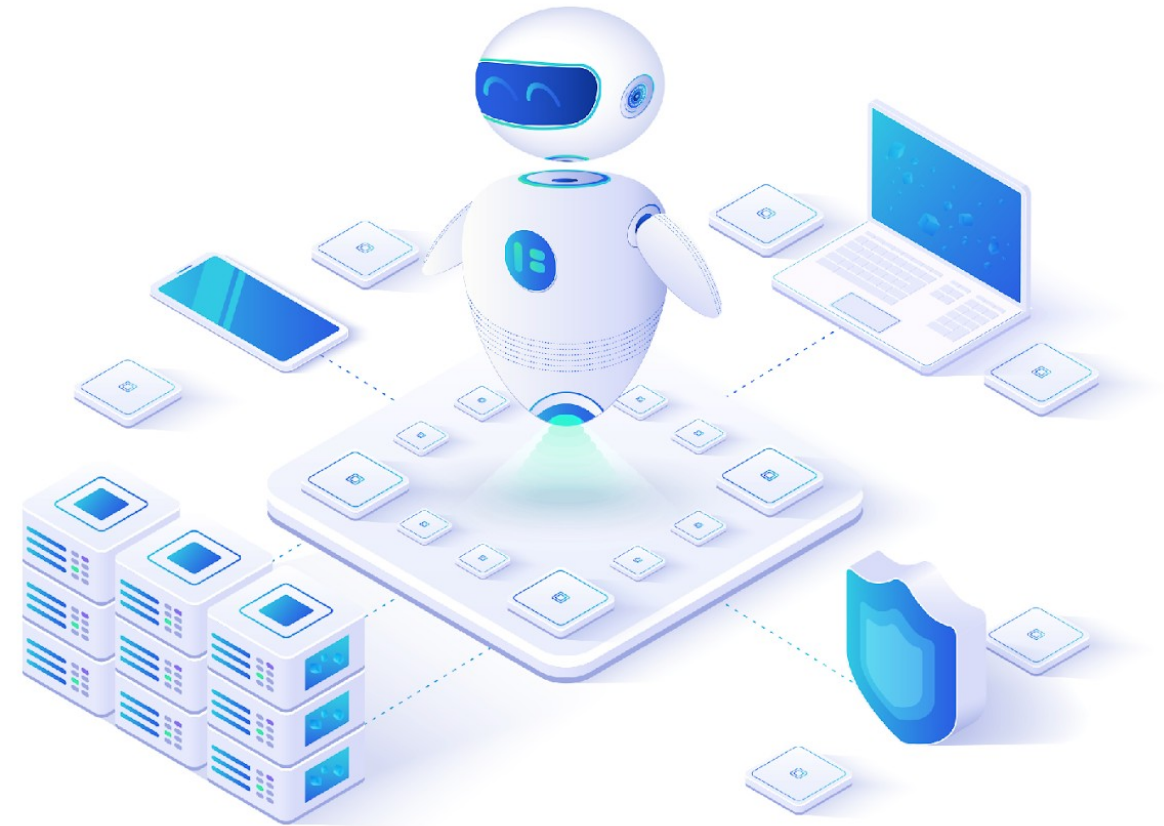
연도	생성형 AI 도입 사례 및 활용 내용	주요 영향 및 변화
2022년	- 12월 OpenAI 챗GPT 출시 및 금융권 관심 급증	- 생성형 AI 기술 첫 등장, 금융업계의 디지털 트랜스포메이션 가속화
	- 초기에는 내부 직원 업무 지원, 문서 요약, 보고서 작성 보조에 활용	- 업무효율성과 정확성 향상 가능성 인식 시작
2023년	- KB금융 'KB-GPT' 업무지원 AI뱅크 도입	- 금융기관 내부보고서 작성, 상품 홍보문구 작성 등 업무 효율성 증대
	- 금융권 AI 고객상담센터(AICC)에서 챗봇 고도화 및 고객 서비스 개선	- 고객응대 자동화로 고객만족도 및 신뢰도 증대
	- 미국 JP모건체이스 400여개 업무에 생성형 AI 적용	- 자산관리, 재무예측, 이상거래 탐지 등 업무 효율성 증대
	- 캐피탈원 등 사기 탐지 시스템에 생성형 AI 도입	- 사기거래 탐지 정확도 향상 및 리스크 관리 강화
2024년	- 국내 NH농협, 신한은행 등 대출심사 및 추천서비스에 AI 활용 확대	- AI 금융상품 추천, 기업 대출 심사 등 금융상품 개발 및 고객 맞춤형 서비스 강화
	- 신한라이프 등 보험사 AI 조직 구축 및 DX 추진	- 금융상품 개발 및 고객 맞춤형 서비스 강화
	- JP모건체이스 400개 이상 AI 활용 사례 확대	- 업무 자동화와 고객 맞춤형 자산관리 고도화
	- 금융권 AI 플랫폼 구축 통한 상용·오픈소스 AI 전략적 활용	- AI 도입 범위 확대 및 디지털 금융 채널 변화
	- 금융산업 내 AI 거버넌스 강화, 규제 개선 필요성 부각	- AI 영업 리스크, 정보보호, 윤리 문제 대응 체계 마련
2025년	- 생성형 AI 활용 금융산업 혁신 가속화 전망	- 디지털금융 개인화, 자동화 심화, 금융산업 생태계 변화
	- AI 기반 리스크 관리 및 고객 상담 고도화 지속	- 고객 서비스 품질 향상 및 금융사고 예방 가능

• 워크플로우 로직



II GenOS 리소스 활용

1. GenOS API
2. VectorDB
3. Util
4. 에이전트 개발



API 상세

Embedding Serving API

텍스트를 벡터로 변환하는 임베딩 서빙. OpenAI Embeddings 규격과 호환됩니다.

- Endpoint : POST {genos_url}/api/gateway/rep/serving/{serving_id}/v1/embeddings
- Headers : Authorization: Bearer {genos_token} < 모델 서빙 상세 > 인증 키
- 모델 정보 : GET .../v1/models → data[0].id (모델명), max_model_len (최대 입력 토큰)

Parameter	Type	Required	Description
input	list[str] str	Y	임베딩할 문자열. 리스트 권장. 한 번에 여러 문장 배치 입력 가능
model	str	N	미지정 시 해당 서빙에 배포된 모델 사용 (예: bge-m3)

Output

- JSON(OpenAI 규격). data[i].embedding 에 입력 순서대로 벡터(list[float]). 실측 1024차원 (서빙 10)

⚠ VDB 검색용 질의 임베딩은 VDB 적재에 쓴 것과 같은 임베딩 서빙이어야 합니다

1. GenOS API

임베딩 서빙 호출 예시

requests / OpenAI SDK: 두 방식 모두 가능

```
import requests
genos_url, serving_id, token = "https://genos.genon.ai", 10, "-인증키-"
headers = {"Authorization": f"Bearer {token}"}
base = f"{genos_url}/api/gateway/rep/serving/{serving_id}"

# 1) 모델 정보: 모델명과 최대 입력 토큰
models = requests.get(f"{base}/v1/models", headers=headers, timeout=10).json()
print(models["data"][0]["id"], models["data"][0]["max_model_len"])

# 2) 임베딩: 배치 입력
texts = ["현대캐피탈 자동차 할부 금리", "오늘 점심 뭐 먹지"]
res = requests.post(f"{base}/v1/embeddings", headers=headers,
                    json={"input": texts}, timeout=30).json()
vectors = [d["embedding"] for d in res["data"]] # 입력 순서 그대로
print(len(vectors), len(vectors[0]))          # 2 1024

# 3) OpenAI SDK: base_url 과 api_key 만 교체
from openai import OpenAI
client = OpenAI(base_url=f"{base}/v1", api_key=token)
res = client.embeddings.create(model="bge-m3", input=texts)
print(len(res.data[0].embedding), res.usage)
```

상세 설명

규격이 OpenAI 와 같다

기존 OpenAI 코드에서 base_url 을 .../rep/serving/{id}/v1 로, api_key 를 인증키로 바꾸면 그대로 동작합니다.

배치 입력

input 에 리스트를 넘기면 한 요청으로 여러 문장을 임베딩합니다. data[i] 가 input[i] 에 대응합니다.

차원·토큰 상한

/v1/models 의 max_model_len 이 입력 1건당 토큰 상한입니다. 넘으면 에러가 납니다.

material

GenOSEmbedding.py. call() · call_batch() · models()

임베딩 서빙: 인증 키 생성

서비스 화면

모델

트레이너 >

평가 >

가드레일 >

배포

도구 >

서빙 >

모델 서빙

코드 서빙

서비스

에이전트 >

애플리케이션 >

운영

승인 >

사용자 페이지

모델 서빙 목록

서빙 생성 + 임베딩

ID	서빙 유형
606	임베딩
578	임베딩
575	임베딩
541	임베딩
361	임베딩
223	임베딩
133	임베딩

모델

트레이너 >

평가 >

가드레일 >

배포

도구 >

서빙 >

모델 서빙

코드 서빙

서비스

에이전트 >

애플리케이션 >

운영

임베딩 모델 서빙 상세

기본 정보 리비전 목록 인증 키 이용 로그 컨테이너 상태

인증 키 목록

인증 키 생성 +

ID	인증 키	승인 상태	허용 IP
744	4b16656c65...	생성결재	-
647	-	생성신청	-
625	b289bf14f1b...	생성결재	-
163	1ed5a9dbe5...	생성결재	-

상세 설명

- [임베딩 모델 확인]**
배포 ≡ 서빙 ≡ 모델 서빙에서 배포된 임베딩 모델 ID 기록
- [인증 키 발급]**
임베딩 모델 서빙 상세 ≡ 인증키 ≡ 인증키 생성을 통해 Bearer Token 발급

API 상세

LLM Serving API

OpenAI Chat Completions 규격. 뒷단이 vLLM 서빙이라 vLLM 이 지원하지 않는 옵션은 무시되거나 에러가 납니다.

- Endpoint : POST {genos_url}/api/gateway/rep/serving/{serving_id}/v1/chat/completions
- Headers : Authorization: Bearer {genos_token} ← 모델 서빙 상세 > 인증 키 생성 후 승인
- 모델 정보 : GET .../v1/models → data[0].id 가 모델명. body 의 model 은 생략 가능 (서빙에 모델이 묶여 있음)

Parameter	Type	Required	Description
messages	list[dict]	Y	[[{role, content}] role: system / user / assistant. VLM 은 content 를 배열로 {type: image_url} 포함
stream	bool	N	SSE 스트리밍. delta.content 가 답변, 추론 모델은 delta.reasoning(_content) 가 생각 조각
temperature · max_tokens	float · int	N	추론 모델은 생각 토큰도 max_tokens 에 포함되므로 넉넉히
response_format	dict	N	구조화 출력 {type: json_schema, ...} 또는 {type: json_object}
tools	list[dict]	N	Function Calling 도구 정의

Output

- choices[0].message.content. stream=True 면 data: 청크의 choices[0].delta.content, 종료는 data: [DONE]
- 오류는 HTTP 오류 코드 또는 {code, errMsg} 봉투로 올 수 있음. 응답에 choices 가 없으면 code / errMsg 를 확인

LLM 서빙 호출 예시 (스트리밍 · 추론 · 이미지)

스트리밍 파싱: 생각(reasoning)과 답변(content)을 나눠 받기

```
import requests, json
url = f"{genos_url}/api/gateway/rep/serving/{serving_id}/v1/chat/completions"
body = {"messages": [{"role": "system", "content": "한 문장으로 답해."},
                     {"role": "user", "content": "17 곱하기 23은?"}],
        "temperature": 0.3, "max_tokens": 1024, "stream": True}

res = requests.post(url, headers=headers, json=body, stream=True, timeout=60)
for raw in res.iter_lines():
    line = raw.decode().strip()
    if not line.startswith("data:"): # ": keep-alive" 주석 등은 건너뛰기
        continue
    chunk = line[5:].strip()
    if chunk == "[DONE]":
        break
    delta = json.loads(chunk)["choices"][0].get("delta", {})
    if delta.get("reasoning") or delta.get("reasoning_content"):
        print("[생각]", delta.get("reasoning") or delta["reasoning_content"])
    if delta.get("content"):
        print(delta["content"], end="", flush=True)

# 이미지 입력(VLM): content 를 배열로
{"role": "user", "content": [
    {"type": "image_url", "image_url": {"url": "data:image/png;base64,..."}},
    {"type": "text", "text": "이 이미지를 설명해줘"}]}
```

상세 설명

OpenAI SDK 도 그대로

OpenAI(base_url=.../v1, api_key=인증키) 후
chat.completions.create(stream=True) 로 호출하면 SSE 파싱을 SDK 가
해줍니다.

추론 모델

delta.reasoning 또는 delta.reasoning_content 로 생각 조각이 옵니다
(서빙마다 키 이름이 다름). 채팅앱에 노출하려면 <think>...</think> 로 감싸
token 으로 보냅니다.

이미지(VLM)

파일을 data:image/png;base64,... 문자열로 바꿔 image_url 에 넣습니다.
VLM 서빙 ID 로 호출해야 합니다.

material

GenOSLLMServing.py. call() · call_with_body(stream=True) ·
make_openai_json_schema()

LLM 서빙: 인증 키 생성

서비스 화면

평가

가드레일

백포

도구

서빙

모델 서빙

코드 서빙

서비스

에이전트

애플리케이션

운영

모델 서빙 목록

서빙 생성 +

전체 서빙

제목

ID	서빙 유형	제목	상세 설명
592	LLM	kimi k2 thinking	
590	LLM	qwen3 next	
589	LLM	gpt-oss 120b (Reasoni...	
588	LLM	deepSeek-V3.1	
587	LLM	Qwen3-235B-A22B-T...	

기본 정보

리비전 목록

인증 키

이용 로그

컨테이너 상

외부 API 추가 +

서빙 리비전 추가 +

인증 키 목록

인증 키 생성 +

ID

검색어를 입력하세요

ID	인증 키	승인 상태	허용 IP	만료 일자	분당 요청...	사용자	요
796	a854ecb1a...	생성결재	-	2027-01-13 00:00:00			

전체 1개 중 1 ~ 1 표시 / 페이지 표시 행수:

10

상세 설명

- **[LLM ID 확인]**
배포 ≡ 서빙 ≡ 모델 서빙에서 사용할 호출할 LLM의 ID를 기록
- **[인증키 생성]**
LLM 모델 서빙 상세 ≡ 인증 키 ≡ 인증 키 생성 후 승인하여 인증 키를 생성

Workflow Serving API (Flowise · 코드서빙 워크플로우 공통)

- Endpoint : POST {genos_url}/api/gateway/workflow/{workflow_id}/run/v2
- Headers : Authorization: Bearer {genos_token} · x-genos-session-id: {chatId} (멀티턴)
- 헬스체크 : GET .../healthcheck → {"status": "ok"}
- 멀티턴 자동 처리는 Flowise 워크플로우만.** chatId-세션 헤더를 받아 히스토리를 자동으로 잊습니다.
 - Python Step 안의 로직과 코드서빙 워크플로우는 세션 ID 를 키로 히스토리를 직접 저장·주입해야 합니다 (3. Util Redis)

Parameter	Type	Required	Description
question	str	Y	사용자 입력. 어드민 「검증」 버튼은 "__verify__" 를 보냄
chatId	str	N	세션 ID (uuid4). 헤더 x-genos-session-id 에도 같은 값 (실측: 둘 다 넣어 멀티턴 확인). Flowise 는 자동 처리, 코드서빙 워크플로우는 직접 구현
stream	bool	N	true 면 줄 앞머리가 token: / result: 인 라인 스트림 (SSE data: 형식이 아님)
overrideConfig	dict	N	워크플로우 변수(vars) 등 실행 설정 재정의

Output

- {"code": 0, "errMsg": "success", "data": {"text", "sourceDocuments", "chatId"}}

▲ 오류도 HTTP 200 으로 옵니다. 반드시 code == 0 을 확인

워크플로우 호출 예시 (멀티턴 · 스트리밍)

같은 세션 ID 로 두 번 호출: 두 번째 질문이 첫 답을 기억한다

```
import requests, json, uuid
url = f"{genos_url}/api/gateway/workflow/{workflow_id}"
session_id = str(uuid.uuid4())
headers = {"Authorization": f"Bearer {token}", "x-genos-session-id": session_id}

print(requests.get(f"{url}/healthcheck", headers=headers).json()) # {"status": "ok"}
```

일반 호출

```
res = requests.post(f"{url}/run/v2", headers=headers, timeout=120,
    json={"question": "1+1은?", "chatId": session_id}).json()
assert res["code"] == 0, res["errMsg"]
print(res["data"]["text"], len(res["data"].get("sourceDocuments") or []))
```

스트리밍: 줄 앞머리 token: / result: (SSE 가 아니라 라인 스트림)

```
res = requests.post(f"{url}/run/v2", headers=headers, stream=True, timeout=120,
    json={"question": "방금 답에 3을 더하면?", "chatId": session_id, "stream": True})
final = None
for raw in res.iter_lines():
    line = raw.decode() if raw else ""
    if line.startswith("token: "): print(line[7:], end="", flush=True)
    elif line.startswith("result: "): final = json.loads(line[8:])
print("\n최종:", final and final.get("text"))
```

상세 설명

세션 = chatId + 헤더

body 의 chatId 와 x-genos-session-id 헤더에 같은 uuid 를 넣으면 앱 채팅과 같은 방식으로 히스토리가 이어집니다. 이는 Flowise 워크플로우가 자동으로 해주는 일이고, 코드서빙 워크플로우는 이 값을 키로 히스토리를 직접 보관해야 합니다.

스트림 형식 주의

워크플로우 스트림은 token: / result: 접두어 라인입니다. 코드서빙 채팅 규약 (data: {"event": "...})과 다릅니다.

검증 버튼이 보내는 것

{"question": "__verify__", "overrideConfig": {}, "history": []}. 코드서빙 워크플로우라면 이 값에 code 0 으로 응답해야 합니다.

material

GenOSWorkflowServing.py. call() · stream_call() · new_session()

서빛 —화면

홈 > 에이전트 > 워크플로우

워크플로우 목록

워크플로우 생성 +

☐ 리소스 사용

ID	제목	상세 설명
3832	Material	

워크플로우 상세

[기본 정보](#)
[리비전 정보](#)
[리비전 이력](#)
[인증 키](#)
[APIM](#)
[이용 로그](#)

인증 키 목록

인증 키 생성 +

ID	인증 키	승인 상태	허용 IP
832	f8143c1bcd7... 	생성결재	-

전체 1개 중 1 ~ 1 표시 / 페이지 표시 행수: 10 ▼

상세 설명

- **[워크플로우 ID 확인]**
서비스 ≡ 에이전트 ≡ 워크플로우에서 호출할 워크플로우의 ID 기록
- **[인증 키 생성]**
워크플로우 상세 ≡ 인증 키 ≡ 인증 키 생성 후 승인하여 인증 키를 생성

API 상세

Code Serving API

- 코드서빙 컨테이너가 노출한 엔드포인트를 게이트웨이 경유로 호출합니다. path 는 컨테이너 서비스 탭에서 설정한 값입니다.

 - Endpoint : POST {genos_url}/api/gateway/code_serving/{code_serving_id}/{revision_id}/{path} 리비전 생략 = 대표 리비전 (가중치 0 이면 503)
 - Headers : Authorization: Bearer {genos_token} < 코드서빙 상세 > 인증 키
 - 헬스체크 : GET .../health → {"status": "ok"} (startupProbe 가 호출. 없으면 배포 실패)

Parameter	Type	GenOS 가 보내는가 / 구현	Description (path 별)
question	str	항상 / 필수	/chat. 사용자 입력. HITL 턴은 빈 문자열. 검증 시 "__verify__" → LLM 없이 code 0 응답
stream	bool	채팅앱 true · 어드민 테스트는 없음 / 두 분기 모두	/chat. true: SSE data: {"event": token sourceDocuments action error end}. 없음·false: JSON 봉투
humanInput	dict	HITL 턴에만 / HITL 쓰면 필수	/chat. 2턴째에 question 은 "" 로 오고 이 값만 옴. 상태 보관은 직접 구현 (섹션 III)
name · arguments	str · dict	항상 / 필수	/mcp/call. 응답 {"code": 0, "data": {"content": [{"type": "text", "text"}]}}
file_path · params · file_content_base64	str · dict · str	항상 / base64 만 필수	/preprocess. 응답 {"code": 0, "data": [청크...]} (다음 명세)

Output

- 성공 code == 0. 실패도 HTTP 200 + code != 0 이므로 반드시 code 확인

코드서빙 호출 예시 (/chat 비스트리밍 · SSE)

/chat: JSON 봉투와 SSE 이벤트 스트림

```
import requests, json
url = f"{genos_url}/api/gateway/code_serving/{code_serving_id}"
headers = {"Authorization": f"Bearer {token}"}
print(requests.get(f"{url}/health", headers=headers).json())    # {"status": "ok"}

# 비스트리밍: {"code": 0, "data": {"text": ., "sourceDocuments": [...]}}
res = requests.post(f"{url}/chat", headers=headers, timeout=120,
                    json={"question": "36개월 할부 금리는?"}.json())
assert res["code"] == 0, res.get("errMsg")
print(res["data"]["text"])

# 스트리밍: data: {"event": ., "data": .} 프레임
res = requests.post(f"{url}/chat", headers=headers, stream=True, timeout=120,
                    json={"question": "36개월 할부 금리는?", "stream": True})
for raw in res.iter_lines():
    line = raw.decode() if raw else ""
    if not line.startswith("data: "):
        continue
    ev = json.loads(line[6:])
    if ev["event"] == "token":      print(ev["data"], end="", flush=True)
    elif ev["event"] == "sourceDocuments": print("\n출처:", [d["metadata"]["file_name"] for d in ev["data"]])
    elif ev["event"] == "action":    print("\nHITL 대기:", ev["data"]["interactionId"])
    elif ev["event"] == "error":     print("\n오류:", ev["data"])
    elif ev["event"] == "end":      break
```

상세 설명

두 가지 응답 형식

stream 이 없으면 JSON 봉투, true 면 SSE. 어드민 워크플로우 테스트·검증 버튼은 stream 없이 호출하므로 두 분기 모두 구현해야 합니다.

이벤트 5종

token(본문) · sourceDocuments(출처) · action(HITL) · error · end. result 이벤트는 핸들러가 없어 무시됩니다.

HITL 2번째

사용자가 선택하면 {"question": "", "humanInput": {...}} 로 다시 호출됩니다. 원 질문이 안 오므로 세션 ID 로 상태를 보관해야 합니다.

코드서빙이 직접 구현해야 하는 것

비스트리밍·SSE 두 응답 분기, __verify__ 즉답, 멀티턴 히스토리(세션 ID 키), HITL 상태 보관, sourceDocuments 구성.

(Flowise 워크플로우에서는 플랫폼이 해주던 일입니다. 템플릿: material/GenOSCodeServing_main.py)

GENOS

홈 > 서빙 > 코드 서빙

코드 서빙 목록

코드 서빙 생성 +

전체 언어

리소스 사용 중인 보기

ID

제목

상세 설명

107

Test Title

106

GenOS Code S...

105

코드 서빙 생성 확인

104

테스트 - 수동(이벤트)

git 동기화 확인서 정보

101

테스트 - 수동(git)

생성

100

테스트 - 수동(GitHub)

생성 테스트 - 외부 연결

99

이벤트 수동용 코드서빙

98

GitLab과 연결 테스트 Title

생성

코드 서빙 생성

제목*

교육용 코드 서빙

상세 설명

상세 설명을 입력하세요

관리 그룹*

GenOS 엔지니어 임시 그룹

저장소 유형*

내부 Gitea (기본)

플랫폼 내장 Gitea에 저장소 자동 생성

외부 Git

외부 Git 저장소 URL + 크레덴셜 연결

닫기

생성

상세 설명

코드 서빙 생성

관리자 페이지 > 서빙 > 코드 서빙 > 코드 서빙 생성 버튼 클릭

코드 서빙 생성 상세

"제목": 코드 서빙 제목

"상세 설명": 코드 서빙 목적 및 설명

"관리 그룹": 코드 서빙을 관리할 그룹

"저장소 유형 - 내부 Gitea": 플랫폼 내장 저장소 가 자동 생성되어 별도 설정 없이 사용 가능

"저장소 유형 - 외부 git": GitHub, GitLab, Gitea 등 외부 저장소와 연결

GENON

- 21 -

GenOS

서비스 화면

코드 서빙 상세

기본 정보

연결 관계

리버전 목록

커밋 목록

인증 키

이용 로그

데이터 소스

제작자

이상민-관리자 sangmin.jee-admin

관리 그룹

GENON AI Engineer

등록일시

2026-09-07 11:32:26

저장소

llmops/634

코드서빙 연결

미연결

저장소 URL

http://llmops-gitea-service:3000/llmops/634.git

이관 수신 URL

https://genos.genon.ai/api/admin/gitea/llmops/634

확장 서비스

API 호출

API

제목*

교육자료 작성용 (내부git)

상세 설명

상세 설명을 입력하세요

트래픽 분배 설정

가중치 합산 기반으로 분배 비율이 계산됩니다. weight=0인 리버전에는 트래픽이 전달되지 않습니다.

ID

커밋 해시

트래픽 분배

분배 비율

해당 정보가 없습니다

수정

코드 서빙 배포 이력

ID

커밋 해시

승인 상태

요청자

요청 일시

승인자

승인/반려 일시

상세

상세 설명

코드 서빙 상세 요소

"저장소 URL": 플랫폼 내부 gitea에 생성된 코드 저장소 주소

코드 서빙: 저장소 불러오기

서비스 화면

```
[genos@sangmin.lee-admin workspace]$ git clone http://llmops-gitea-service:3000/llmops/634.git
Cloning into '634'...
Username for 'http://llmops-gitea-service:3000': sangmin.lee-admin
Password for 'http://sangmin.lee-admin@llmops-gitea-service:3000':
```

```
[genos@sangmin.lee-admin workspace]$ git clone http://llmops-gitea-service:3000/llmops/634.git
Cloning into '634'...
Username for 'http://llmops-gitea-service:3000': sangmin.lee-admin
Password for 'http://sangmin.lee-admin@llmops-gitea-service:3000':
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
[genos@sangmin.lee-admin workspace]$ cd 634
```

상세 설명

• 코드 서빙 저장소 Clone

1. 터미널에서 코드 서빙 저장소 주소로 git clone 실행
2. Username에 GenOS 아이디 입력
3. Password에 GenOS 패스워드 입력

코드 서빙: 저장소 업데이트

서비스 화면

```
● [genos@sangmin.lee-admin 634]$ git push origin
Username for 'GenOS': sangmin.lee-admin
Password for 'GenOS':
✓ Identity verified as sangmin.lee-admin
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 104 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 2.53 KiB | 432.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote: . Processing 1 references
remote: Processed 1 references in total
To http://llmops-gitea-service:3000/llmops/634.git
    7efdbc4..18517a1  main -> main
○ [genos@sangmin.lee-admin 634]$
```

상세 설명

- 코드 서빙 저장소에 push

생성한 파일을 코드 서빙 저장소에 git push

저장소에 반영된 코드를 기준으로 배포 진행 가능

서비스 화면

코드 서빙 상세

기본 정보 | 연결 관계 | 리비전 목록 | 커밋 목록 | 인증 키 | 이용 로그 | 데이터 소스

제작자: 이상민-관리자 sangmin.lee-admin | 관리 그룹: GENON AI Engineer | 등록일시: 2026-09-07 11:32:26

저장소: llmops/634 | 코드서빙 연결: 미연결

저장소 URL: http://llmops-gitea-service:3000/llmops/634.git

이관 수신 URL: https://genos.genon.ai/api/admin/gitea/llmops/634

자동 생성됩니다. 이 URL 을 개별명 이관 신청 시 입력하세요.

확장 서비스

API 호출: API

코드 서빙 배포 이력

ID	커밋 해시	승인 상태	요청자	요청 일시	승인자
----	-------	-------	-----	-------	-----

리비전 추가

도커 이미지*: 도커 이미지를 선택하세요

인스턴스 타입*: 인스턴스 타입을 선택하세요

GPU 할당량: GPU 할당량을 선택하세요

0

복제본*: 1

브랜치*: main

커밋 해시*: 커밋 해시를 선택하세요

7efdbc43 Initial commit - llmops

닫기

상세 설명

- **코드 서빙 배포**
관리자 페이지 > 서빙 > 코드 서빙 > 리비전
추가 버튼 클릭
- **코드 서빙 배포 상세**
"도커 이미지" : 코드 실행에 사용할 컨테이너
실행 환경 선택
"인스턴스 타입" : 배포 서버의 CPU, 메모리 등
실행 사양 설정
"GPU 할당량" : 모델 추론 또는 연산에 사용할
GPU 자원 설정
"복제본" : 동시에 실행할 서버 인스턴스 개수
"브랜치" : 배포에 사용할 코드 브랜치
"커밋 해시" : 특정 코드 버전을 기준으로 배포

서비스 화면

코드 서빙 상세

+ 리비전 추가

+ 카탈로그 등록

기본 정보

연결 관계

리비전 목록

커밋 목록

인증 키

이용 로그

데이터 소스

ID ▾

Q 검색어를 입력하세요

☐ 리소스 사용 중인 목록만 보기

ID ▾	커밋 해시	제작자	등록일시 ▾	연결 상태	배포 상태
3865	7efdbc43e477	이상민-관리자 sangmin.lee-admin	2026-09-07 14:48:53	● 오프라인	미배포

10 ▾

개씩 노출

1-1 / 총 1개

페이지

< 1 >

코드 서빙 리비전 상세

기본 정보

컨테이너 상태

환경 설정

컨테이너 서비스

제작자

이상민-관리자 sangmin.lee-admin

등록일시

2026-09-07 14:48:53

배포 상태

미배포

언어 사용

REV ID

3865

커밋 해시

7efdbc43e477600518686649c22cde18d31ce796

도커 이미지

mnc/template-code-serving:latest

언어

Python

복제본

1

인스턴스 타입

smicro (1 CPU Core, 1 Gb Memory)

GPU 할당량

사용하지 않음 0

실행 확장 (HA) 다음 배포 시 적용

사용

확장 범위

동시 실행 인스턴스 수

배포

취소

상세 설명

- 코드 서빙 리비전 배포**
리비전을 추가하였다면
리비전 목록 > 리비전 선택 > 배포 클릭

서비스 화면

승인 목록

ID ▾

Q 검색어를 입력해 주세요

Q 승인 유형 ▾

Q 결재 상태 ▾

승인

반려

☐	ID ▾	승인 유형 ▾	상세 유형 ▾	대상 ▾	상태 ▾	요청자 ▾	요청 일시 ▾	승인자 ▾	승인/반려 일시 ▾	승인	반려
☐	47356	배포	코드 서빙	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	-	-	✓ 승인	✕ 반려
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☒	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	이상민-관리자 sangmin.lee-admin	2026-09-07 14:51:50	☐	☐
☐	47356	배포	배포	교육자료 작성용 (내부git)	배포						

상세 설명

- 코드 서빙 배포 승인
관리자 페이지 > 승인 > 승인 버튼 클릭

서비스 화면

상세 설명

- 코드 서빙 상세 (컨테이너 서비스)
관리자 페이지 > 서빙 > 코드 서빙 >
리비전 목록 > 리비전 선택 > 컨테이너 서비스 >
샘플 코드 보기 버튼 클릭

1. GenOS API

코드 서빙 저장소 업데이트 (1)

서비스 화면

샘플 코드 보기

워크플로우/컨테이너/MCP 각 컨테이너 서비스 타입별 Python 샘플 코드입니다. 복사 후 main.py 에 붙여넣어 사용하세요.

워크플로우 sample 컨테이너 sample MCP sample

복사

```

"""
GenOS Python code-serving - 워크플로우 sample.
/chat 만 호출. 외부 LLM 의존성 없이 question 을 echo 로 응답.
requirements.txt: fastapi, uvicorn[standard]
"""
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/chat")
def chat(body: dict):
    question = (body or {}).get("question") or ""
    if question == "__verify__":
        return {"code": 0, "data": {"text": "verified"}}
    if not question:
        return {"code": 0, "data": {"text": "[ERROR] question is empty"}}
    # 외부 LLM 의존성 없이 echo. 실제 코드서빙에선 원하는 로직으로 교체.
    return {"code": 0, "data": {"text": f"echo: {question}"}}

```

main

상세 설명

- 샘플 코드 보기**
 워크플로우 연동에 필요한 샘플 코드 확인
 샘플 코드를 main.py에 복사해 코드 서빙 실행
 파일로 사용 가능
- 코드 스페이스로 이동**
 main.py 파일 생성을 위해 코드 스페이스로 이동

1. GenOS API

코드 서빙 저장소 업데이트 (2)

서비스 화면

The screenshot shows a code editor with a file explorer on the left. The file explorer shows a project structure with a 'main.py' file highlighted. The main editor displays the content of 'main.py', which is a FastAPI application. A terminal window on the right shows the execution of a 'git push' command, which is successful. The terminal output includes the username 'sangmin.lee-admin' and the repository path 'http://llmops-gitea-service:3000/llmops/634.git'.

```

634 > main.py
1  import os
2  from contextlib import asynccontextmanager
3
4  import httpx
5  from fastapi import FastAPI, HTTPException, Request
6  from fastapi.responses import StreamingResponse
7
8  GENOS_URL = os.getenv("GENOS_URL", "https://genos.genon.ai").rstrip("/")
9  REP_SERVING_ID = os.getenv("REP_SERVING_ID", "742")
10 MODEL = os.getenv(
11     "MODEL",
12     "/model/genos/9eb2daaa8597bf192a8b0e73f848f3a102794df5",
13 )
14
15 @asynccontextmanager
16 async def lifespan(app: FastAPI):
17     app.state.http = httpx.AsyncClient(timeout=60)
18     yield
19     await app.state.http.aclose()
20
21 app = FastAPI(lifespan=lifespan)
22
23 @app.get("/health")
24 def health():
25     return {"status": "ok"}
26
27 @app.post("/chat")
28 async def chat(request: Request, body: dict):
29     question = (body or {}).get("question") or ""
30     stream = bool((body or {}).get("stream"))
31
32     if question == "__verify__":
33         return {"code": 0, "data": {"text": "verified"}}
34
35     if not question:
36         return {"code": 0, "data": {"text": "[ERROR] question is empty"}}
37
38     token = "a9424cc871b34fa1be2110208b14a1d"
39     if not token:
40         raise HTTPException(500, "GENOS_BEARER_TOKEN is not configured")
41
42
43
44
45

```

```

[genos@sangmin.lee-admin 634]$ git push origin
Username for 'GenOS': sangmin.lee-admin
Password for 'GenOS':
✓ Identity verified as sangmin.lee-admin
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 104 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 2.53 KiB | 432.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote: . Processing 1 references
remote: Processed 1 references in total
To http://llmops-gitea-service:3000/llmops/634.git
7efdbc4..18517a1  main -> main

```

상세 설명

- 코드 서빙 저장소에 push
복사한 샘플코드를 main.py 파일에 붙여넣고,
코드 서빙 저장소에 git push

서비스 화면

코드 서빙 상세

기본 정보

연결 관계

리비전 목록

커밋 목록

인증 키

이용 로그

데이터 소스

제작자

이상민-관리자 sangmin.lee-admin

관리 그룹

GENON AI Engineer

등록일시

2026-09-07 11:32:26

저장소

llmops/634

코드서빙 연결

미연결

저장소 URL

http://llmops-gitea-service:3000/llmops/634.git

이관 수신 URL

https://genos.genon.ai/api/admin/gitea/llmops/634

자동 생성됩니다. 이 URL 을 개별당 이관 신청 시 입력하세요.

확장 서비스

API 호출

API

코드 서빙 배포 이력

ID	커밋 해시	승인 상태	요청자	요청 일시	승인
----	-------	-------	-----	-------	----

제목*

교육자료 작성용 (내부git)

상세 설명

상세 설명을 입력하세요

트래픽 분배 설정

가중치 합산 기반으로 분배 비율이 계산됨

ID

커밋 해시

리비전 추가

도커 이미지*

도커 이미지를 선택하세요

인스턴스 타입*

인스턴스 타입을 선택하세요

GPU 할당량

GPU 할당량을 선택하세요

0

복제본*

1

브랜치*

main

커밋 해시*

커밋 해시를 선택하세요

18517a1e

시연용 - 이상민-관리자

7efdbc43

Initial commit - llmops

닫기

상세 설명

- 코드 서빙 배포**
 관리자 페이지 > 서빙 > 코드 서빙 >
 리비전 추가 버튼 클릭
- 코드 서빙 배포 상세**
 배포할 도커 이미지, 인스턴스 타입 등을 설정
 main.py 가 포함된 커밋 해시를 기준으로 배포
 버전 지정

서비스 화면

코드 서빙 상세

기본 정보 데이터 소스 환경 설정 커밋 목록 인증 키 컨테이너 상태 **컨테이너 서비스**

중지

컨테이너 서비스

이 코드서빙을 워크플로우/MCP/전처리 등 다른 곳에서 사용 가능하게 노출합니다.

</> 샘플 코드 보기

☐ 워크플로우로 사용

채팅 서비스의 워크플로우 드롭다운에 노출됩니다.

☐ 전처리기로 사용

데이터 전체리기 목록에 노출되어 문서 인덱싱 시 사용할 수 있습니다.

☐ MCP 서버로 사용

MCP 서버로 노출되어 워크플로우/Flowise/외부 MCP 클라이언트에서 호출할 수 있습니다. 도구 목록 실행은 코드서빙이 노출하는 두 path 용단으로 결정됩니다.

저장

상세 설명

- **코드 서빙 워크플로우로 사용 등록 (1)**
관리자 페이지 > 서빙 > 코드 서빙 >
컨테이너 서비스 > 워크플로우로 사용 클릭

1. GenOS API

코드 서빙 컨테이너 서비스 등록 (2)

서비스 화면

코드 서빙 상세

기본 정보

데이터 소스

환경 설정

커밋 목록

인증 키

컨테이너 상태

컨테이너 서비스

배포

중지

컨테이너 서비스

이 코드서빙을 워크플로우/MCP/전처리기 등 다른 곳에서 사용 가능하게 노출합니다.

워크플로우로 사용

재팅 서비스의 워크플로우 드롭다운에 노출됩니다.

재팅 호출 경로*

/chat

검증 성공 — 엔드포인트가 응답합니다.

재팅에서 이 코드서빙을 호출할 때 사용할 path. 코드 안에 POST <경로> 를 구현해야 합니다.

전처리기로 사용

데이터 전처리기 목록에 노출되어 문서 인덱싱 시 사용할 수 있습니다.

MCP 서버로 사용

MCP 서버로 노출되어 워크플로우/Flowise/외부 MCP 클라이언트에서 호출할 수 있습니다. 도구 목록 실행은 코드서빙이 노출하는 두 path 응답으로 결정됩니다.

샘플 코드 보기

검증

저장

상세 설명

코드 서빙 워크플로우로 사용 등록 (2)

검증 버튼 클릭하여 '/chat' 엔드포인트 정상 응답 여부 확인 > 저장 버튼 클릭

서비스 화면

워크플로우 목록

제목 ▼ 🔍 코드 ✕ ☐ 리소스 사용 중인 목록만 보기 (?) ☐ 전용 Flowise 워크플로우만 보기 (?)

[카탈로그 보기](#)

+ 워크플로우 생성

ID	제목	상세 설명	타인	제작자	관리 그룹	등록일시	연결 상태	배포 상태
6190	(CS ID=556 REV=3376) 교육용 ... ● 저작	Code-serving: 코드 서빙 스트리밍 테스트	코드서빙 기반	이상민-관리자 sangmin.lee-admin	GENON AI Engineer	2026-08-27 17:52:39	● 온라인	배포 완료

상세 설명

- **코드 서빙 워크플로우로 등록 완료**
'서비스>에이전트>워크플로우'로 페이지를 이동하여 코드서빙으로 생성한 워크플로우를 확인하실 수 있습니다.

1. GenOS API

코드 서빙 워크플로우 테스트

서비스 화면

워크플로우 상세

배포

중지

재배포

테스트

기본 정보

인증 키

이용 로그

컨테이너 상태

A2A 에이전트

플러그인

데이터 소스

제작자

이상민-관리자 sangmin.lee-admin

관리 그룹

GENON AI Engineer

등록일시

2026-08-27 17:52:39

제목

(CS ID=556 REV=3376) 교육용 코드서빙 워크플로우

배출 일시

2026-09-02 13:23:23

연결된 리소스

0개

API 호출

</> API 사용

상세 설명

Code-serving: 코드 서빙 스트리밍 테스트

워크플로우 타입

코드서빙 기반

배포 상태

배포 완료

메인 프로세스 실행 상태

배포 이력

ID	리비전 아이디
18274	82978
18070	82978

10

개씩 노출

1-2 / 총 2개

워크플로우 테스트

워크플로우*

(CS ID=556 REV=3376) 교육용 코드서빙 워크플로우 / Code-servir

워크플로우

워크플로우를 선택하세요

입력 json

1 [{"question": "Type Your Message"}]

닫기

테스트

리비전 아이디

중지일시

요청 수

인-관리자

min.lee-admin

검색어를 입력해 주세요

2026-09-02 13:23:23

0

2026-09-01 17:28:52

0

페이지

<

1

>

상세 설명

코드 서빙 워크플로우 테스트

코드 서빙 워크플로우를 클릭하여 테스트 진행

테스트 시 해당 워크플로우는 자동 선택

입력 JSON 안에 "question" key 의 인자로

질문을 작성하여 테스트

코드 서빙 워크플로우 테스트

서비스 화면

워크플로우 테스트

전체 초기화

A

```

{
  "text": "안녕하세요! 반가워요. 무엇을 도와드릴까요?",
  "is_updatable": true
}

```

B

워크플로우 테스트입니다

워크플로우*

(CS ID=556 REV=3376) 코드 서빙 스트리밍 테스트 / Code-serving

워크플로우

워크플로우를 선택하세요

입력 json

```

1 {"question": "안녕"}

```

닫기

테스트

상세 설명

- 코드 서빙 워크플로우 테스트
A 화면에 답변 확인 가능

서비스 화면

코드 서빙 상세

기본 정보

연결 관계

리버전 목록

커밋 목록

인증 키

이동 로그

데이터 소스

제작자

이상민-관리자 sangmin.lee-admin

관리 그룹

GENON AI Engineer

등록일시

2026-08-27 16:00:13

저장소

l1mops/556

코드서빙 연결

미연결

저장소 URL

http://l1mops-gitea-service:3000/l1mops/556.git

이관 수신 URL

https://genos.genon.ai/api/admin/gitea/l1mops/556

자동 생성됩니다. 이 URL 을 개발명 이관 신청 시 입력하세요.

확장 서비스

API 호출

API

코드 서빙 배포 이력

ID	커밋 해시	승인 상태	요청자	요청 일시
3376	e8024c6			

코드 서빙 API 사용

기본 엔드포인트

코드 서빙 엔드포인트는 아래 형식을 따릅니다.

https://genos.genon.ai/api/gateway/code_serving/556/{사용자_정의_경로}

{사용자_정의_경로} 예는 코드 서빙에서 구현한 HTTP 경로를 입력합니다.

예: '/predict', '/inference', '/health'

인증

모든 요청에 'Authorization' 헤더를 포함해야 합니다.

import requests

serving_id = 556

bearer_token = 'GenOS에서 발급한 인증키'

genos_url = 'https://genos.genon.ai'

base_url = f'{genos_url}/api/gateway/code_serving/{serving_id}'

headers = {

"Authorization": f"Bearer {bearer_token}",

"Content-Type": "application/json"

}

API 호출 예시

코드 서빙에서 구현한 경로에 따라 아래와 같이 요청합니다.

코드 서빙에서 구현한 URL 경로를 지정합니다.

user_path = '/predict'

res = requests.post(f'{base_url}{user_path}', headers=headers, json={

"input": "예시 입력 데이터"

})

print(res.json())

curl 예시

curl -X POST https://genos.genon.ai/api/gateway/code_serving/556/{사용자_정의_경로} -H "Authorization: Bearer <인증키>" -H "Content-Type: application/json" -d '{"input": "예시 입력 데이터"}'

닫기

복사

상세 설명

- 코드 서빙 API 호출
- 코드 서빙 상세 페이지 내 API 호출을 클릭하여 코드서빙 API 사용법 확인 가능

서비스 화면

```
In [2]: import requests

serving_id = 
bearer_token = 
genos_url = 'https://genos.genon.ai'

base_url = f"{genos_url}/api/gateway/code_serving/{serving_id}"
headers = {
    "Authorization": f"Bearer {bearer_token}",
    "Content-Type": "application/json"
}
# 코드 서빙에서 구현한 URL 경로를 지정합니다.
user_path = '/chat'

res = requests.post(f"{base_url}{user_path}", headers=headers, json={
    "question": "안녕"
})
print(res.json())

{'code': 0, 'data': {'text': '안녕하세요! 무엇을 도와드릴까요? 궁금한 점이 있거나 도움이 필요하시면 언제든지 말씀해 주세요.'}}
```

상세 설명

- 코드 서빙 API 호출

코드서빙 API 사용 방법을 복사하여 Bearer_token과 user_path, body 부분을 코드서빙에서 배포한 형태에 맞게 작성한 후 테스트하실 수 있습니다.

코드 서빙 MCP 서버로 사용

서비스 화면

```

main.py

from datetime import datetime, timezone

def ok(data):
    return {"code": 0, "errMsg": "success", "data": data}

def fail(msg):
    return {"code": 1, "errMsg": str(msg), "data": None}

@app.post("/mcp/list")
def mcp_list():
    return ok({
        "tools": [{
            "name": "now",
            "description": "현재 UTC 날짜와 시간을 ISO 8601 형식으로 반환합니다.",
            "inputSchema": {"type": "object", "properties": {}},
        }]
    })

@app.post("/mcp/call")
def mcp_call(body: dict | None = None):
    if (body or {}).get("name") != "now":
        return fail(f"unknown tool: {(body or {}).get('name', '')}")

    now = datetime.now(timezone.utc).isoformat()
    return ok({"content": [{"type": "text", "text": now}]})

```

상세 설명

- 코드 서빙 MCP 사용 등록

샘플 코드 보기 안에 mcp 부분을 복사하여 '/mcp/list', 'mcp/call' 을 작성한 후 git push 까지 진행합니다.

코드 서빙 MCP 호출

서비스 화면

코드 서빙 리비전 상세

배포 중지

기본 정보 컨테이너 상태 환경 설정 컨테이너 서비스

컨테이너 서비스
이 코드서빙을 워크플로우/MCP/전처리기 등 다른 곳에서 사용 가능하게 노출합니다.

샘플 코드 보기

☐ 워크플로우로 사용
재팅 서비스의 워크플로우 드롭다운에 노출됩니다.

☐ 전처리기로 사용
데이터 전처리기 목록에 노출되어 문서 인덱싱 시 사용할 수 있습니다.

☒ MCP 서버로 사용
MCP 서버로 노출되어 워크플로우/Flowise/외부 MCP 클라이언트에서 호출할 수 있습니다. 도구 목록-실행은 코드서빙이 노출하는 두 path 응답으로 결정됩니다.
도구 목록 path (tools/list)*
/mcp/list
코드 안에 POST <경로> 를 구현해 {name,description,inputSchema,...} 를 반환해야 합니다.
도구 실행 path (tools/call)*
/mcp/call
코드 안에 POST <경로> 를 구현해 {name, arguments} 를 받고 {content:[{type:"text",text:"..."}]} 를 반환해야 합니다.

외부 호출 주소는 저장 후 자동 발급됩니다 — Flowise/외부 MCP 클라이언트는 이 주소를 등록해 MCP 서버를 사용합니다.

저장

상세 설명

- 코드 서빙 MCP 사용 등록
리비전 배포까지 완료 되었다면 mcp/list 와 mcp/call 경로를 설정한 후 저장 버튼을 누릅니다.

상세 설명

+ MCP 서버 생성

ID	제목	타입	MCP 도구	제작자	관리 그룹	등록일시	연결 상태	배포 상태
715								실행 준비 중
712								배포 완료
709								-
706								-
700	(CS ID=556 REV=3538) 교육용 코드서빙	코드서빙 기반	-	이상민-관리자 sangmin.lee-admin	GENON AI Engin...	2026-09-02 14:18:10	● 온라인	배포 완료
697								배포 완료
694								미배포
691								미배포
688								미배포
685								배포 완료

10개씩 노출1-10 / 총 308개

페이지<12345...31

배포 > 도구 > MCP 서버 목록에서 배포된 MCP
서버 확인

1. GenOS API

코드 서빙 MCP 서버 호출

서비스 화면

MCP 서버 상세

기본 정보

데이터 소스

행위 유형

인증 키

제작자

이상민-관리자 sangmin.lee-admin

등록일시

2026-09-02 14:18:10

배포 상태

배포 완료

메인 프로세스 실행 상태

배포 이력

ID

리버전 제목

10

개씩 노출

0-0 / 총 0개

MCP 서버 테스트

now

parameter

type

value

해당 정보가 없습니다

결과

1

2

3

"2026-09-07T04:52:27.576228+00:00"

닫기

테스트

테스트

수정

리버전 제목

검색어를 입력해 주세요

승인자

승인/반려 일시

보기

페이지

1

상세 설명

코드 서빙 MCP 사용 등록

테스트 버튼을 클릭하고 테스트 할 도구를 설정 후 테스트 버튼을 클릭하면 도구를 사용한 결과를 확인하실 수 있습니다.

1. GenOS API

코드 서빙 MCP 서버 호출

서비스 화면

```
import requests

serving_id = [redacted]
bearer_token = [redacted]
genos_url = 'https://genos.genon.ai'

base_url = f"{genos_url}/api/gateway/code_serving/{serving_id}"
headers = {
    "Authorization": f"Bearer {bearer_token}",
    "Content-Type": "application/json"
}
# 코드 서빙에서 구현한 URL 경로를 지정합니다.
user_path = '/mcp/list'

res = requests.post(f"{base_url}{user_path}", headers=headers, json={})
print(res.json())

{'code': 0, 'errMsg': 'success', 'data': {'tools': [{'name': 'now', 'description': '현재 UTC 날짜와 시간을 ISO 8601 형식으로 반환합니다.', 'inputSchema': {'type': 'object', 'properties': {}}]}}}
```

```
import requests

serving_id = [redacted]
bearer_token = [redacted]
genos_url = 'https://genos.genon.ai'

base_url = f"{genos_url}/api/gateway/code_serving/{serving_id}"
headers = {
    "Authorization": f"Bearer {bearer_token}",
    "Content-Type": "application/json"
}
# 코드 서빙에서 구현한 URL 경로를 지정합니다.
user_path = '/mcp/call'

res = requests.post(f"{base_url}{user_path}", headers=headers, json={'name': 'now', 'arguments': {}})
print(res.json())

{'code': 0, 'errMsg': 'success', 'data': {'content': [{'type': 'text', 'text': '2026-09-07T04:45:58.182829+00:00'}]}}
```

상세 설명

- 코드 서빙 MCP API 사용

Bearer_token과 user_path, body안에 도구 이름과 arguments를 입력하여 코드 서빙의 MCP를 api로 확인하실 수 있습니다.

코드 서빙 전처리기 사용 등록

서비스 화면

```

main.py

@app.post("/preprocess")
def preprocess(body: dict):
    body = body or {}
    if body.get("file_path") == "__verify__":
        return ok({})
    if not body.get("file_path") and not body.get("file_content_base64"):
        return ok({})

    params = body.get("params") or {}
    chunk_size = int(params.get("chunk_size", 1000))
    overlap = int(params.get("chunk_overlap", 100))

    buf, name = _load_file(body)
    if not buf:
        return fail("no file_path / base64")

    ext = os.path.splitext(name)[1].lower()
    reg_date = datetime.now().isoformat() + "Z"
    properties = []

    if ext == ".pdf":
        reader = PdfReader(BytesIO(buf))
        pages_text = [(p.extract_text() or "") for p in reader.pages]
        n_page = len(pages_text)
        all_chunks = [_chunk_text(t, chunk_size, overlap) for t in pages_text]
        total = sum(len(c) for c in all_chunks) or 1
        gid = 0
        for pid, chunks in enumerate(all_chunks):
            page_no = pid + 1
            for cid, chunk in enumerate(chunks):
                properties.append({
                    "text": chunk,
                    "n_char": len(chunk),
                    "n_word": len(chunk.split()),
                    "n_line": len(chunk.split("\n")),
                    "i_page": page_no,
                    "e_page": page_no,
                    "n_page": n_page,
                    "i_chunk_on_page": cid,
                    "e_chunk_on_page": len(chunks)
                })

```

상세 설명

- 코드 서빙 전처리기 사용 등록

'/preprocesser' 경로로 샘플 코드에 있는 전처리기 코드를 붙여넣기 한 후 git add , git commit , git push를 진행하고 리비전 배포를 진행합니다.

서비스 화면

기본 정보컨테이너 상태환경 설정컨테이너 서비스

컨테이너 서비스

이 코드서빙을 워크플로우/MCP/전처리기 등 다른 곳에서 사용 가능하게 노출합니다.

워크플로우로 사용

제팅 서비스의 워크플로우 드롭다운에 노출합니다.

전처리기로 사용

데이터 전처리기 목록에 노출되어 문서 인덱싱 시 사용할 수 있습니다.

전처리기 호출 경로*

/preprocess

검증

검증 성공 — 엔드포인트가 응답합니다.

전처리기가 호출할 path, 코드 안에 POST <경로> 를 구현해 {file_path, file_content_base64, params} 를 받고 {code:0, data:[VectorProperties...]} 를 반환해야 합니다.

지원 확장자*

pdf

임의로 구분 (예: hwp, pdf, md, txt, json). 데이터 인덱싱 시 이 확장자에서 본 전처리기가 후보로 노출됩니다.

기본 파라미터 (JSON)

{ "chunk_size": "1000", "chunk_overlap": "100" }

데이터 인덱싱 통해 기본값으로 채워질 JSON. 예: { "chunk_size": "1000", "chunk_overlap": "100" }

MCP 서버로 사용

MCP 서버로 노출되어 워크플로우/Flowise/외부 MCP 클라이언트에서 호출할 수 있습니다. 도구 목록-실행은 코드서빙이 노출하는 두 path 응답으로 결정됩니다.

샘플 코드 보기

저장

상세 설명

코드 서빙 전처리기 사용 등록

배포가 완료되었다면 '/preprocessor' 경로가 정상적으로 작성되었는지 검증 버튼 클릭하여 확인한 후 전처리 할 문서의 확장자와 파라미터를 입력합니다.

모든 설정이 끝났다면 저장 버튼 클릭합니다.

GENON

- 45 -

GenOS

상세 설명

전처리기 목록

ID ↕	제목 ↕	상세 설명	확장자	타입	제작자 ↕	관리 그룹 ↕	등록일시 ↕	연결 상태	배포 상태
787	(CS ID=556 REV=3862) 교육...	Code-serving: 교육용 코드서빙	pdf	코드서빙 기반	이상민-관리자 sangmin.lee-admin	GENON AI Engineer	2026-09-07 14:04:15	● 온라인	<button>배포 완료</button>

- 코드 서빙 전처리기 사용

배포 > 문서 전처리 > 전처리기 페이지로

이동하여 코드 서빙으로 배포한 전처리기 확인

API 상세

Code Serving Preprocessor API

코드서빙으로 배포한 전처리기를 preprocess-api 가 호출하는 계약입니다. 컨테이너 서비스 탭 「전처리기로 사용」으로 등록합니다.

- Endpoint : POST {genos_url}/api/gateway/code_serving/{id}/{rev}/preprocess
- 검증 : file_path = "__verify__" 로 호출 → 즉시 code 0 응답

Parameter	Type	필수	Description
params	dict	N	전처리기 등록 화면에서 입력한 파라미터 (chunk_size, chunk_overlap 등)
file_content_base64	str	Y	파일 내용 base64. 디코드해서 임시 파일로 저장 후 처리. 유일한 파일 입력 경로
file_name	str	N	원본 파일명. 확장자로 로더를 고를 때 사용 (없으면 .pdf 가정)

Output

- { "code": 0, "errMsg": "success", "data": [GenOSVectorMeta, ...] } . 청크 하나가 Weaviate 객체 하나
- GenOSVectorMeta: text · i_page · e_page · n_chunk_of_doc · chunk_bboxes(뷰어 하이라이트) ... extra 허용 → 커스텀 메타데이터 자유

1. GenOS API

코드서빙 전처리기 최소 구현

main.py: /preprocess 엔드포인트 (material/GenOSPreprocessor_main.py 축약)

```
import base64, os, tempfile
from fastapi import FastAPI, Request
from pydantic import BaseModel
app = FastAPI()

class GenOSVectorMeta(BaseModel):
    class Config: extra = "allow" # 커스텀 메타데이터 추가 자유 → Weaviate 프로퍼티가 된다
    text: str = None; i_page: int = None; e_page: int = None
    i_chunk_on_doc: int = None; n_chunk_of_doc: int = None; n_page: int = None
    chunk_bboxes: str = None; media_files: str = None; title: str = None

@app.get("/health")
async def health(): return {"status": "ok"}

@app.post("/preprocess")
async def preprocess(request: Request):
    body = await request.json()
    if body.get("file_path") == "__verify__": # 검증 버튼
        return {"code": 0, "errMsg": "success", "data": []}
    with tempfile.NamedTemporaryFile(suffix=".pdf", delete=False) as tmp:
        tmp.write(base64.b64decode(body["file_content_base64"])) # ★ file_path 로 읽지 않는다
    try:
        vectors = DocumentProcessor()(tmp.name, **(body.get("params") or {}))
        return {"code": 0, "errMsg": "success", "data": vectors}
    except Exception as e:
        return {"code": 1, "errMsg": str(e), "data": None}
```

상세 설명

file_path 로 읽지 말 것

코드서빙 파드에는 문서 NFS 가 없습니다. preprocess-api 가 넣어주는 file_content_base64 를 디코드해 쓰세요.

등록 절차

git push → 리비전 배포 → 컨테이너 서비스 > 전처리기로 사용 > 경로 /preprocess > 검증 > 확장자-파라미터 입력 > 저장 → 배포 > 문서 전처리 > 전처리기 목록에 표시

파라미터

등록 화면에 입력한 값이 params 로 그대로 옵니다. chunk_size 처럼 문서마다 바꿀 값은 코드에 박지 말고 params 로 받으세요.

DocumentProcessor

load_documents(로더) → split_documents(청킹) → compose_vectors(메타) 세 단계. 포맷·청킹·메타 확장 지점이 각각 분리돼 있습니다.

코드 서빙 생성: 외부 깃 연동

서비스 화면

코드 서빙 생성

제목*

교육자료 작성용 (외부 git)

상세 설명

상세 설명을 입력하세요

관리 그룹*

GENON AI Engineer

저장소 유형*

① 운영망에 Gitea가 없는 환경(예: 네트워크 분리 고객)에서는 외부 Git을 선택하세요.
이관 시 외부 Git으로 푸시됩니다.

☐ 내부 Gitea (기본)

플랫폼 내장 Gitea에 저장소 자동 생성

☒ 외부 Git

외부 Git 저장소 URL + 크레덴셜 연결

외부 Git 인증 정보

상세 설명

• 코드 서빙 생성

배포 > 서빙 > 코드서빙 > 코드 서빙 생성을
통해 코드 서빙을 생성

• 저장소 유형 선택

외부 Git을 선택

코드 서빙: 외부 깃 연동

서비스 화면

저장소 유형*

① 운영망에 Gitea가 없는 환경(예: 네트워크 분리 고객)에서는 외부 Git을 선택하세요. 이관 시 외부 Git으로 푸시됩니다.

☐ 내부 Gitea (기본)
플랫폼 내장 Gitea에 저장소 자동 생성

☒ 외부 Git
외부 Git 저장소 URL + 크레덴셜 연결

외부 Git 인증 정보

Git 호스트 *

Github

외부 Git URL *

https://github.com/Kimyongari/test-chatting.git

외부 Git ID *

Kimyongari

크레덴셜 (PAT) *

.....

✓ 검증

닫기

생성

상세 설명

- 외부 Git 인증 정보 입력
Git 호스트, 외부 Git URL, 외부 Git ID, PAT
토큰을 입력

1. GenOS API

코드 서빙: 외부 깃 연동

서비스 화면

코드 서빙 상세

기본 정보 연결 관계 리비전 목록 커밋 목록 인증 키 이용 로그 데이터 소스

제작자 관리 그룹 등록일시 제목*

김용준-관리자 GENON AI Engineer 2026-08-31 16:19:22 교육자료 작성용 (

yongjun.kim-admin

저장소 코드서빙 연결 상세 설명

Kimyongari/test-chatting 미연결 상세 설명을 입력하

저장소 URL*

Git 호스트* Github

외부 Git ID* Kimyongari

확장 서비스

API 호출

API

```
[genos@saegyeol.park-admin 583]$ git clone https://github.com/Kimyongari/test-chatting.git
Cloning into 'test-chatting'...
remote: Enumerating objects: 86, done.
remote: Counting objects: 100% (86/86), done.
remote: Compressing objects: 100% (59/59), done.
remote: Total 86 (delta 15), reused 86 (delta 15), pack-reused 0 (from 0)
Receiving objects: 100% (86/86), 161.62 KiB | 4.49 MiB/s, done.
Resolving deltas: 100% (15/15), done.
[genos@saegyeol.park-admin 583]$
```

상세 설명

- **저장소 URL 복사**
코드 서빙 상세에서 저장소 URL을 복사
- **Git clone**
코드스페이스에서 저장소 URL을 통해 레포지토리를 클론

API 상세

MCP Server API

표준 MCP JSON-RPC 2.0 (streamable HTTP). GenOS 에 등록된 MCP 서버의 도구를 조회·호출합니다.

- Endpoint : POST {genos_url}/api/gateway/mcp/{mcp_server_id}/mcp
- Headers :
 - Authorization: Bearer {genos_token}
 - Accept: application/json, text/event-stream
 - MCP-Protocol-Version: 2025-06-18
- initialize 응답 헤더로 Mcp-Session-Id가 오면, 이후 요청에 같은 이름의 **요청 헤더**로 되실어 보낸다 (MCP 표준. 상태 보관형 서버가 여러 HTTP 요청을 한 연결로 묶는 용도). GenOS 게이트웨이는 발급만 하고 검사하지 않아 initialize 없이 tools/call을 바로 불러도 동작.

Parameter	Type	Required	Description
jsonrpc	str	Y	"2.0"
id	int	Y	요청 ID (호출마다 증가)
method	str	Y	initialize · tools/list · tools/call
params	dict	method별	tools/call: {"name": tool_name, "arguments": {...}} / initialize: {protocolVersion, capabilities, clientInfo}

Output

- tools/list → result.tools[{name, description, inputSchema}] . LLM 이 읽는 도구 설명과 동일
- tools/call → result.content[{type: "text", text}]
- 인증키는 MCP 서버 상세에서 발급 (코드서빙 인증키와 다름)

MCP 서버 호출 예시 (initialize · tools/list · tools/call)

JSON-RPC 호출: 응답이 JSON 이든 SSE 든 같은 함수로 파싱

```
import requests, json
mcp_url = f"{genos_url}/api/gateway/mcp/{mcp_server_id}/mcp"
headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json",
           "Accept": "application/json, text/event-stream",
           "MCP-Protocol-Version": "2025-06-18"}
session = {}

def rpc(method, params=None, rpc_id=1):
    body = {"jsonrpc": "2.0", "id": rpc_id, "method": method, **({"params": params} if params else {})}
    h = {"**headers", **({"Mcp-Session-Id": session["id"]} if session.get("id") else {})}
    res = requests.post(mcp_url, headers=h, json=body, timeout=120)
    if res.headers.get("Mcp-Session-Id"): session["id"] = res.headers["Mcp-Session-Id"]
    text = res.text
    if "text/event-stream" in res.headers.get("content-type", ""): # SSE 면 data: 줄만
        text = next(l[6:] for l in text.splitlines() if l.startswith("data: "))
    out = json.loads(text)
    if "error" in out: raise RuntimeError(out["error"])
    return out["result"]

rpc("initialize", {"protocolVersion": "2025-06-18", "capabilities": {},
                  "clientInfo": {"name": "edu", "version": "1.0"}})
for t in rpc("tools/list", rpc_id=2)["tools"]:
    print(t["name"], "-", t["description"], t["inputSchema"].get("required"))
r = rpc("tools/call", {"name": "add", "arguments": {"a": 1, "b": 2}}, rpc_id=3)
print(r["content"][0]["text"]) # "3"
```

상세 설명

인식되는 프로토콜 버전

2024-11-05 · 2025-03-26 · 2025-06-18 · 2025-11-25 · 2026-07-28. 그 외는 -32022 로 거절됩니다. 헤더와 body _meta 의 버전이 다른면 -32020.

코드서빙 MCP 의 도구 설명

GenOS 는 DB 스냅샷이 아니라 코드서빙 /mcp/list 응답을 그대로 도구 목록으로 씁니다. description 을 바꾸려면 코드를 고쳐 리버전을 재배포합니다.

인증키 두 종류

MCP 서버 상세의 키는 이 /mcp/{id}/mcp 경로용, 코드서빙 상세의 키는 컨테이너 직접 호출(.../code_serving/{id}/mcp/list)용입니다.

material

GenOSMCPServer.py. initialize() · list_tools() · call() · call_text()

MCP 서버 배포

서비스 화면

The image displays three overlapping screenshots of a web application interface, likely for managing a cloud platform (MCP).

Top Screenshot (MCP 도구 상세): Shows the 'MCP 도구 상세' (MCP Tool Details) page. It includes tabs for '기본 정보' (Basic Info), '중요 정보' (Important Info), '리비전 이력' (Revision History), and '리소스 목록' (Resource List). The '기본 정보' tab is active, displaying details for a tool named '도로교통법 법규 검색 도구' (Road Traffic Law Regulation Search Tool). The creator is '김용준-관리자' (Kim Yongjun - Admin), the group is 'MNC DS', and the registration date is '2026-02-27 15:48:07'. There is a '제약*' (Restriction*) field and a '상세 설명' (Detailed Description) field. A '수정' (Edit) button is visible at the bottom right.

Bottom Left Screenshot (MCP 서버 상세): Shows the 'MCP 서버 상세' (MCP Server Details) page. It includes tabs for '기본 정보' (Basic Info), '중요 정보' (Important Info), '리비전 이력' (Revision History), '인증 키' (Authentication Key), and '컨테이너 상태' (Container Status). The '기본 정보' tab is active, displaying details for a server named '도로교통법 법규 검색' (Road Traffic Law Regulation Search). The creator is '김용준-관리자' (Kim Yongjun - Admin), the group is 'MNC DS', and the registration date is '2026-03-03 15:12:38'. There is a '제약*' (Restriction*) field and a '상세 설명' (Detailed Description) field. A '수정' (Edit) button is visible at the bottom right.

Bottom Right Screenshot (인증 키 목록): Shows the '인증 키 목록' (Authentication Key List) page. It includes tabs for '기본 정보' (Basic Info), '중요 정보' (Important Info), '리비전 이력' (Revision History), '인증 키' (Authentication Key), and '컨테이너 상태' (Container Status). The '인증 키' tab is active, displaying a list of authentication keys. The list has columns for 'ID', '인증 키' (Authentication Key), '승인 상태' (Approval Status), and '허용 IP' (Allowed IP). One key is listed with ID '843', value '3879508a...', status '생성결재' (Creation Approval), and IP '-'. A '인증 키 생성 +' (Generate Authentication Key +) button is visible at the top left. A pagination bar at the bottom shows '전체 1개 중 1 ~ 1 표시 / 페이지 표시 행수: 10' (Total 1 item, showing 1 ~ 1 / Page display row count: 10).

상세 설명

- **MCP 도구 생성**
배포 ≡ MCP 도구 ≡ MCP 도구 생성에서
MCP 도구를 생성
- **MCP 서버 생성 및 배포**
배포 ≡ MCP 서버 ≡ MCP 서버 생성 ≡
MCP 서버 상세 ≡ 배포 신청/승인하여 MCP
서버를 배포
- **인증 키 생성**
MCP 서버 상세 ≡ 인증 키 ≡ 인증키
생성/승인을 통해 인증 키 생성

MCP 서버: MCP 도구 호출

서비스 화면

```

1 from dotenv import load_dotenv
2 import os
3 load_dotenv()
4
5 MCP_SERVER_ID = os.getenv('MCP_SERVER_ID')
6 MCP_SERVER_BEARER_TOKEN = os.getenv('MCP_SERVER_BEARER_TOKEN')
7 GENOS_URL = os.getenv('GENOS_URL')
8 from Material.GenOSMCPServer import GenOSMCPServer
9
10 mcp_server = GenOSMCPServer(mcp_server_id=MCP_SERVER_ID,
11                             bearer_token=MCP_SERVER_BEARER_TOKEN, genos_url=GENOS_URL)
12
13 tool_name = 'retriever'
14 arguments = {"query": "도로교통법이 뭐야?", "topk": 1}
15 mcp_server.call(tool_name=tool_name, arguments=arguments)

```

✓ [2] 1초 202밀리초

code_execution: { "code": "from dotenv import load_dotenv\nimport os\nload_dotenv()\n\nMCP_SERVER_ID = os.getenv('MCP_SERVER_ID')\nMCP_SERVER_BEARER_TOKEN = os.getenv('MCP_SERVER_BEARER_TOKEN')\nGENOS_URL = os.getenv('GENOS_URL')\nfrom Material.GenOSMCPServer import GenOSMCPServer\n\nmcp_server = GenOSMCPServer(mcp_server_id=MCP_SERVER_ID,\nbearer_token=MCP_SERVER_BEARER_TOKEN, genos_url=GENOS_URL)\n\ntool_name = 'retriever'\narguments = {'query': '도로교통법이 뭐야?', 'topk': 1}\nmcp_server.call(tool_name=tool_name, arguments=arguments)", "type": "text", "text": "370, \"bbox\": {\"l\": 0.5648094369583772, \"t\": 0.31963951664220425, \"r\": 0.6234837636226366, \"b\": 0.3090991951197421, \"coord_origin\": \"BOTTOMLEFT\"}, \"type\": \"text\", \"ref\": \"#/texts/5621\"}]'\", \"'n_chunk_of_doc\": 477.0, \"'file_ext\": \"'.pdf'\", \"'subject\": \"'본 문서는 **행정안전부**에서 제작(경찰청 소관)한 **도로교통법 시행규칙** 전문으로, 도로교통법 및 동법 시행령에서 위임된 사항과 그 시행에 필요한 세부 규정을 담고 있습니다.\\n\\n주요 내용으로는 **신호기 및 안전표지의 설치·관리 기준**, **보행자 및 차량의 통행 방법**, **속도 제한**, **운전면허 시험 및 발급 절차**, **교통안전교육 운영**, **어린이통학버스 신고 및 안전 관리**, **자동차 운전학원의 등록 및 지정 기준**, **음주운전 방지장치 및 자율주행자동차 관련 규정**, 그리고 **과태료 및 범칙행위 처리 절차** 등을 종합적으로 포괄하고 있습니다.'\", \"'n_word\": 1968.0, \"'text\": \"'HEADER: 도로교통법 시행규칙, [별지 제42호의2서식] 자동차운전면허시험(제1종보통·제2종) 응시원서\\n도로교통법 시행규칙, [별지 제42호의2서식] 자동차운전면허시험(제1종보통·제2종) 응시원서, \\n자동차운전면허시험(제1종보통·제2종) 응시원서\\n* 뒤쪽에 질병·신체에 관한 신고서가 있으므로 질의 내용 확인 후 적어서 함께 제출하여 주시기 바랍니다. \\n\\n접수번호\\n\\n접수일자\\n\\n\"}

상세 설명

- MCP 서버 연결**
 GenOSMCPServer.py 에서 mcp_server_id, bearer_token, genos_url을 입력하여 객체 생성
- MCP 도구 호출**
 tool_name, arguments를 입력하여 배포된 MCP 서버에 존재하는 MCP Tool을 사용할 수 있음

A2A Agent API

- Agent Card : GET {genos_url}/a2a/agents/{agent_card_id}/.well-known/agent.json
- 호출 : POST {genos_url}/a2a/{agent_card_id}
- Headers : Authorization: Bearer {genos_token} ← A2A 에이전트 상세 > 인증 키

Parameter	Type	Required	Description
jsonrpc	str	Y	"2.0"
id	str	Y	요청 식별자
method	str	Y	message/send (단발) · message/stream (SSE). 레거시 tasks/send · tasks/sendSubscribe 도 허용
params.message.parts	list[dict]	Y	[{"type": "text", "text": 질문}] (role: user)
params.contextId	str	N	멀티턴. 이전 응답의 contextId 를 넣음
params.timeout	int	N	응답 대기 초 (기본 600)

Output

- `result.artifacts[*].parts` 에 답변 텍스트 (`status.message` 는 null, `history` 에 대화 기록)

A2A 에이전트 호출 예시

Agent Card 조회 → message/send → 텍스트 추출 (material/GenOSA2A.py)

```
import requests
base = f"{genos_url}/a2a"
headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}

# 1) Agent Card: 이름·설명·스킬
card = requests.get(f"{base}/agents/{agent_card_id}/well-known/agent.json",
                    headers=headers, timeout=30).json()
print(card["name"], [s["name"] for s in card.get("skills", [])])

# 2) 호출: JSON-RPC message/send
body = {"jsonrpc": "2.0", "id": "edu-1", "method": "message/send",
        "params": {"message": {"role": "user",
                                "parts": [{"type": "text", "text": "36개월 할부 금리 알려줘"}]},
                    "timeout": 600}}
res = requests.post(f"{base}/{agent_card_id}", headers=headers, json=body, timeout=620).json()

# 3) 답변 텍스트: status.message.parts 우선, 없으면 artifacts
result = res["result"]
parts = result.get("status", {}).get("message", {}).get("parts") or \
        [p for a in result.get("artifacts", []) for p in a.get("parts", [])]
print("".join(p.get("text", "") for p in parts))
print("contextId:", result.get("contextId")) # 다음 턴에 params.contextId 로 넣으면 멀티턴
```

상세 설명

스트리밍

method 를 message/stream 으로. GenOS 는 event: 줄 없이 data: 만 보내므로 data.result.kind(status-update / artifact-update) 로 구분합니다. GenOSA2A.call_agent_stream().

멀티턴

응답 result.contextId 를 다음 요청 params.contextId 에 넣습니다.

API 상세

Service Router API

여러 서빙 리비전을 고정 엔드포인트 하나로 묶고 가중치·카나리·Shadow·서킷브레이커 정책으로 라우팅합니다. 호출 코드는 바뀌지 않습니다.

- Endpoint : POST {genos_url}/api/gateway/service-router/{endpoint}/{path}
- Headers : Authorization: Bearer {router_token} ← 라우터 인증키 (서빙 인증키 아님). 허용 IP·만료·호출 한도를 라우터 단위로 관리
- {endpoint} : 라우터 생성 시 지정한 식별자 (영문 소문자·숫자·하이픈, 예: hc-llm)

라우터 유형	path	인증	규약
서빙. LLM / VLM	/v1/chat/completions	라우터 키	OpenAI Chat. LLM 서빙과 body 동일
서빙. Embedding	/v1/embeddings	라우터 키	OpenAI Embeddings. 임베딩 서빙과 동일
서빙. Rerank	/v1/rerank	라우터 키	리랭킹 (body 형식 미검증)
코드 서빙	{코드서빙이 정의한 path}	라우터 키	코드서빙 path 그대로 (예: /chat)

Output

- 응답 model 필드로 실제 라우팅된 리비전의 모델 확인. 카나리 중이면 호출마다 달라질 수 있음

▲ 라우터 경유 시 라우터의 가드레일만 적용(대상 서빙 자체 가드레일 무시). 코드서빙 가드레일은 라우터 플러그인이 유일한 적용 지점

서비스 라우터 호출 예시

URL 만 라우터로 바꾸면 끝: body 는 대상 서빙과 동일

```
import requests
router_endpoint, router_token = "hc-llm", "-라우터 인증키-"
url = f"{genos_url}/api/gateway/service-router/{router_endpoint}"
headers = {"Authorization": f"Bearer {router_token}"}
```

LLM 라우터: 2장 LLM 서빙 body 그대로

```
body = {"messages": [{"role": "system", "content": "한국어 한 문장으로 답해."},
                    {"role": "user", "content": "서비스 라우터가 뭐야?"},
                    {"temperature": 0.3, "max_tokens": 512}]
res = requests.post(f"{url}/v1/chat/completions", headers=headers, json=body, timeout=60).json()
print(res["choices"][0]["message"]["content"])
print("실제 응답 모델:", res.get("model")) # 라우터가 고른 리비전. 카나리면 호출마다 다를 수 있음
```

임베딩 라우터

```
res = requests.post(f"{url}/v1/embeddings", headers=headers,
                    json={"input": ["현대캐피탈 오토론"]}, timeout=30).json()
```

코드서빙 라우터: 코드서빙이 정의한 path 그대로

```
res = requests.post(f"{url}/chat", headers=headers,
                    json={"question": "안녕!"}, timeout=120).json()
```

상세 설명

무엇을 얻나

리비전 교체·A/B·카나리·Shadow(같은 요청을 다른 리비전에도 복제해 결과 비교)·서킷브레이커를 호출 코드 변경 없이 라우터 정책으로 처리합니다.

인증키가 다르다

라우터 인증키는 「배포 > 서비스 라우터 > 상세 > 인증 키」에서 발급합니다. 서빙 인증키로는 401 입니다.

가드레일 주의

라우터를 거치면 라우터에 붙인 가드레일만 적용됩니다. 대상 서빙에 붙인 가드레일은 무시되므로 라우터 플러그인 탭에서 다시 설정하세요.

사전 준비

배포 > 서비스 라우터 생성(유형·endpoint) → 타깃 리비전 추가·가중치 → 정책·플러그인 → 인증 키 생성 → 승인

벡터 DB 생성 및 문서 업로드

서비스 화면

벡터 DB 생성

제목*
TestVDB

상세 설명
상세 설명을 입력하세요

관리 그룹*
MNC DS

데이터 형식*
☒ 문서
 ☐ QA (Deprecated)
 ☐ 자동적재
 ☐ FAQ

수동 소키마 ☐

닫기 생성

문서 추가

1 단계 2 단계

파일 업로드*

업로드 중...

도로교통법 시행규칙(행정안전부령)(제00610호)(20260224).pdf | 18.6 MB | 1.4 Mbps | 42% uploading

도로교통법(법률)(제21016호)(20260101).pdf | 466.7 KB | 0% uploading

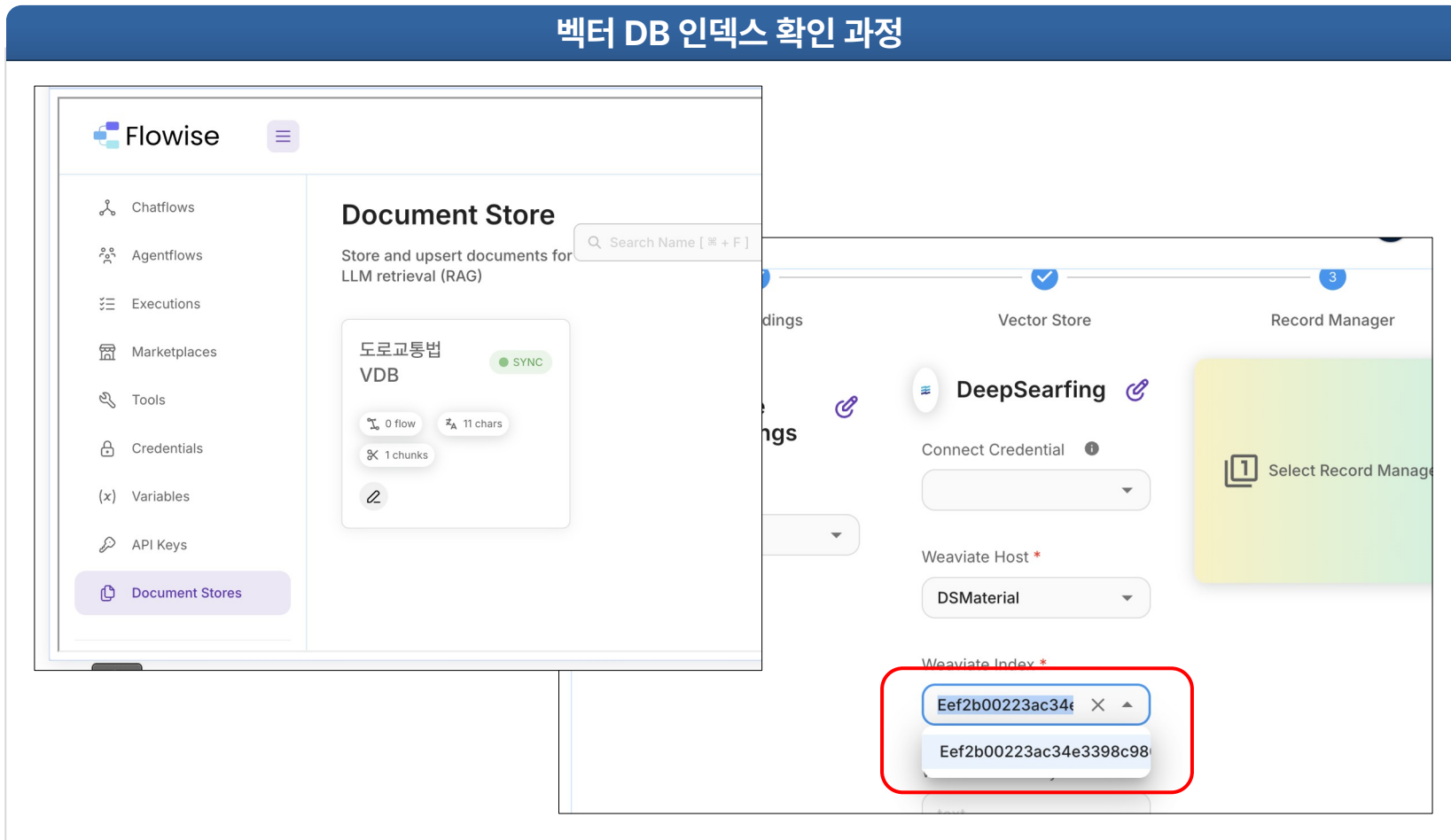
도로교통법 시행령(대통령령)(제35947호)(20260102).pdf | 1.2 MB | 0% uploading

닫기 이전 생성

상세 설명

- **벡터 DB 생성**
개발 ≡ 데이터 ≡ 벡터 DB → 벡터 DB 생성을 통해 벡터 DB를 생성
- **문서 적재**
벡터 DB 상세 ≡ 문서 목록 ≡ 업로드를 통해 문서를 적재

벡터 DB 인덱스 확인 과정



상세 설명

- **Document Store 생성**
워크플로우 ≡ 워크플로우 상세 ≡ Flowise 메뉴 ≡ Document Store에서 Document Store 생성
- **VDB Index 확인**
Options → Upsert Chunks → Vector Store → Weaviate Index에서 생성한 VDB의 Index 확인

API 상세

Weaviate 연결 정보 (VDB · AI 드라이브)

VDB(AI 드라이브 포함)는 Weaviate 컬렉션 하나입니다. weaviate-client v4 또는 REST/GraphQL 로 직접 검색합니다.

- 주소 : [llmops-weaviate-service](#) HTTP 8080 / gRPC 50051 (내부망. 코드스페이스·코드서빙에서만 접근)
- 인증 : Authorization: Bearer {vdb_key} ← 벡터DB 상세 > 인증 키. **해당 컬렉션에만 권한 · 익명 접근 불가 (필수)**
- 인덱스 : 벡터DB 상세의 index 문자열 = Weaviate 컬렉션명 (GET /v1/schema 로 이 키가 볼 수 있는 컬렉션 확인)

Method	Parameters	Description
dense_search	query, topk, user_level	벡터 유사도 (near_vector)
bm25_search	query, topk, user_level	키워드. 암호화 적재 VDB 는 text 가 암호문이라 결과 없음
hybrid_search	query, topk, alpha, user_level, file_name_pattern, folder_path	벡터+키워드. alpha 0=키워드만 1=벡터만. 파일명·드라이브 폴더 필터 가능
to_source_documents	rows	검색 결과 → 채팅 sourceDocuments 규약 (text 외 프로퍼티 전부 metadata 로)

Output

- list[dict] 청크 프로퍼티. text · file_name · i_page · e_page · doc_id · security_level · chunk_bboxes · ai_drive_node_path · is_encrypted ...

⚠ AI 드라이브의 경우 GenOS 앱 채팅은 사용자 등급으로 security_level 필터를 자동으로 걸지만 직접 검색은 user_level 을 직접 넣어야 합니다

2. Vector DB

벡터 DB 검색 예시 (Read Key · hybrid · 보안등급 필터)

```
import requests, weaviate
from weaviate.classes.init import Auth
from weaviate.classes.query import Filter, MetadataQuery

vdb_key = "-VDB 읽기 키-" # ★ 벡터DB 상세 > 인증 키 (없으면 401)
vdb_index = "K6960b01d5fff43779aa20707d51f902f"
question, user_level = "근로자의 정의", 2

# 1) 질문 임베딩: VDB 적재에 쓴 것과 같은 임베딩 서빙
vec = requests.post(f"{genos_url}/api/gateway/rep/serving/{emb_id}/v1/embeddings",
                    headers={"Authorization": f"Bearer {emb_token}"},
                    json={"input": [question]}).json()["data"][0]["embedding"]

# 2) hybrid + 보안등급 필터 (+ 파일명 필터 = Filter RAG)
client = weaviate.connect_to_custom(http_host="llmops-weaviate-service", http_port=8080, http_secure=False,
                                   grpc_host="llmops-weaviate-service", grpc_port=50051, grpc_secure=False,
                                   auth_credentials=Auth.api_key(vdb_key))
col = client.collections.get(vdb_index)
res = col.query.hybrid(query=question, vector=vec, alpha=0.5, limit=5,
                       filters=Filter.by_property("security_level").less_or_equal(user_level)
                       & Filter.by_property("file_name").like("*근로*"),
                       return_metadata=MetadataQuery(score=True))
for o in res.objects:
    p = o.properties
    print(f"[{o.metadata.score:.3f}] {p['file_name']} p[{p['i_page']}]", (p.get("text") or "")[:60])
client.close()
```

상세 설명

Read Key 발급 위치

벡터DB 상세 > 인증 키 탭 (READ 권한). 재발급하면 기존 키가 rotate 되어 즉시 무효화됩니다.

암호화 VDB

is_encrypted 가 true 면 text 가 암호문입니다. AES-256-GCM / PBKDF2-SHA256(salt=genos) 로 복호화. 키는 운영자에게 받아 Env 로 주입 (genos-api-v1.ipynb §6).

커스텀 토큰 제약

커스텀 토큰(형태소 역인덱스)은 암호화 적재 VDB 에서 동작하지 않습니다. 벡터 검색만 사용.

material

GenOSVectorDB.py. vdb_key 필수 · hybrid_search(user_level, file_name_pattern, folder_path) · to_source_documents()

2. Vector DB

벡터 DB 호출

서비스 화면

```

1 from Material.GenOSVectorDB import GenOSVectorDB
2 from dotenv import load_dotenv
3 import os
4
5 load_dotenv()
6
7 GENOS_URL = os.getenv("GENOS_URL", "")
8 EMBEDDING_SERVING_ID = os.getenv("EMBEDDING_SERVING_ID", "")
9 EMBEDDING_BEARER_TOKEN = os.getenv("EMBEDDING_BEARER_TOKEN", "")
10 WEAVIATE_URL = os.getenv("WEAVIATE_URL", "")
11 WEAVIATE_INDEX = os.getenv("WEAVIATE_INDEX", "")
12 WEAVIATE_HTTP_PORT = os.getenv("WEAVIATE_HTTP_PORT", "")
13 WEAVIATE_GRPC_PORT = os.getenv("WEAVIATE_GRPC_PORT", "")
14
15 vdb = GenOSVectorDB(weaviate_ip = WEAVIATE_URL,
16                     http_port = WEAVIATE_HTTP_PORT,
17                     grpc_port = WEAVIATE_GRPC_PORT,
18                     idx = WEAVIATE_INDEX,
19                     embedding_serving_id = EMBEDDING_SERVING_ID,
20                     embedding_bearer_token = EMBEDDING_BEARER_TOKEN,
21                     embedding_genos_url = GENOS_URL
22                     )
23
24 vdb.show_documents()

```

✓ [64] 422밀리초

weaviate에 접속하였습니다.
VDB를 설정했습니다.
유사도 검색을 위한 임베딩 모델로 10번 임베딩 모델을 사용합니다.
embedding model: 10번과 토큰이 입력되었습니다.
['도로교통법 시행규칙(행정안전부령)(제00610호)(20260224).pdf',
'도로교통법(법률)(제21016호)(20260101).pdf',
'도로교통법 시행령(대통령령)(제35947호)(20260102).pdf']

상세 설명

VDB 호출

GenOSVectorDB(vdb_key, idx, embedding_serving_id, embedding_bearer_token, genos_url) 로 객체를 만들면 적재 문서를 확인·검색할 수 있습니다. weaviate_host · 포트는 기본값 (llmops-weaviate-service, 8080/50051)입니다.

Weaviate 는 익명 접근이 꺼져 있어 벡터DB 상세 > 인증 키에서 발급한 VDB 전용 Read Key 가 없으면 401 입니다. 루트 키가 아니라 해당 컬렉션에만 권한이 있는 키입니다.

서비스 화면

```
1 from Material.GenOSVectorDB import GenOSVectorDB
2 from dotenv import load_dotenv
3 import os
4
5 load_dotenv()
6
7 GENOS_URL = os.getenv("GENOS_URL", "")
8 EMBEDDING_SERVING_ID = os.getenv("EMBEDDING_SERVING_ID", "")
9 EMBEDDING_BEARER_TOKEN = os.getenv("EMBEDDING_BEARER_TOKEN", "")
10 WEAVIATE_URL = os.getenv("WEAVIATE_URL", "")
11 WEAVIATE_INDEX = os.getenv("WEAVIATE_INDEX", "")
12 WEAVIATE_HTTP_PORT = os.getenv("WEAVIATE_HTTP_PORT", "")
13 WEAVIATE_GRPC_PORT = os.getenv("WEAVIATE_GRPC_PORT", "")
14
15 vdb = GenOSVectorDB(weaviate_ip = WEAVIATE_URL,
16                     http_port = WEAVIATE_HTTP_PORT,
17                     grpc_port = WEAVIATE_GRPC_PORT,
18                     idx = WEAVIATE_INDEX,
19                     embedding_serving_id = EMBEDDING_SERVING_ID,
20                     embedding_bearer_token = EMBEDDING_BEARER_TOKEN,
21                     embedding_genos_url = GENOS_URL
22                     )
23
24 vdb.hybrid_search(query = '안녕?')
```

✓ [61] 792밀리초

```
{
  'i_page': 367.0,
  'file_path': '/nfs-root/docs/document-100610.pdf',
  'media_files': [
    {
      'name':
        'image_000355_f38035d960709812b2d85f86d973bcc9838fa47594377b522f352233b6cbe70c.png',
      'type': 'image',
      'ref': '#/pictures/355'}],
  'n_words': None,
  'file_hash': UUID('d75b41bc-a222-c1ae-a364-5299f48ce23a'),
  'text': 'HEADER: 도로교통법 시행규칙, [별지 제42호의2서식] 자동차운전면허시험(제1종보통·제2종) 응시원서\n도로교통법 시행규칙, [별지 제42호의2서식] 자동차운전면허시험(제1종보통·제2종) 응시원서, \n자동차운전면허시험(제1종보통·제2종) 응시원서\n\n위쪽에 질병·신체에 관한 신고서가 있으므로 질의 내용을 확인 후 적어서 함께 제출하여 주시기 바랍니다. \n접수번호\n접수일자\n|
```

```
15 vdb = GenOSVectorDB(weaviate_ip = WEAVIATE_URL,
16                     http_port = WEAVIATE_HTTP_PORT,
17                     grpc_port = WEAVIATE_GRPC_PORT,
18                     idx = WEAVIATE_INDEX,
19                     embedding_serving_id = EMBEDDING_SERVING_ID,
20                     embedding_bearer_token = EMBEDDING_BEARER_TOKEN,
21                     embedding_genos_url = GENOS_URL
22                     )
23 result = vdb.search_from_file_name(query = '안녕?',
24                                   file_name_pattern = "*도로교통법*")
25 result
26 result = [{ 'text' : i['text'][:100] + '...', 'file_name' : i['file_name']} for i in result]
27 result
```

🔍 [9]

뉴스는 검색을 위한 검색 모델도 10만 검색 모델을 사용합니다.
embedding model: 10번과 토큰이 입력되었습니다.

```
[{ 'text': 'HEADER: 도로교통법 시행규칙, [별지 제20호의3서식] 어린이통학버스 안전교육 통지서\n도로교통법 시행규칙, [별지 제20호의3서식] 어린이통학버스 안전교육 통지서, \n어린이통...',
  'file_name': '도로교통법 시행규칙(행정안전부령)(제00610호)(20260224).pdf'},
{ 'text': 'HEADER: 도로교통법 시행규칙, [별지 제63호서식] 삭제\n도로교통법 시행규칙, [별지 제63호서식] 삭제, \n[별지 제63호서식] 삭제 <2007.4.27>\n\n도로교통법 시...',
  'file_name': '도로교통법 시행규칙(행정안전부령)(제00610호)(20260224).pdf'},
{ 'text': 'HEADER: 도로교통법 시행규칙, [별지 제42호의2서식] 자동차운전면허시험(제1종보통·제2종) 응시원서\n도로교통법 시행규칙, [별지 제42호의2서식] 자동차운전면허시험(제1종보통·제2종) 응시원서, \n도로교통법 시행규칙, [별지 제144호의2서식] 운전경력증명서 발급 신청서, \n운전경력증명...',
  'file_name': '도로교통법 시행규칙(행정안전부령)(제00610호)(20260224).pdf'}]
```

상세 설명

문서 검색

hybrid_search(query, topk, alpha). 벡터+키워드 혼합. alpha 0=키워드만, 1=벡터만.

보안등급 필터

user_level=2 처럼 넘기면 security_level <= 2 문서만 검색됩니다. 앱 채팅이 자동으로 하는 필터를 직접 검색에서는 직접 걸어야 합니다.

파일명·폴더 필터 (Filter RAG)

file_name_pattern="*근로*" 로 특정 파일만, folder_path="/2002/3499/" 로 AI 드라이브 특정 폴더 아래만 검색합니다. file_name 은 word 토큰나이저라 언더스코어 등에서 토큰이 끊깁니다.

API 상세

MinIO Service (cdn-api)

클러스터 MinIO 에 파일을 올려 임시 공개 URL(Presigned URL)을 받습니다. 이 URL 을 답변 마크다운데 넣으면 채팅 화면에서 이미지·차트가 렌더됩니다.

- Endpoint : POST <http://lmops-cdn-api-service:8080/minio/upload/temp> (내부망 · 인증 없음)
- Method : multipart/form-data
- 기존 객체 URL : POST .../minio/presigned/create json {hostname, bucket_name, object_name}

Parameter	Type	Required	Description
file	file	Y	업로드할 파일 (이미지 · html 차트 · csv 등)
hostname	str	Y	사용자가 접속하는 GenOS 도메인. presigned URL 의 호스트로 쓰임 (예: https://genos.genon.ai)

Output

- {"code": 0, "data": {"presigned_url": "https://.../temp/<uuid>.png?X-Amz-..."}}. 파일명은 UUID 로 바뀌고 URL 에 만료가 있음
- 채팅 노출: 이미지는 ![제목](presigned_url) 마크다운데 token 에 실어 보냄. HTML 차트는 링크로

MinIO 업로드 · 차트 노출 예시

matplotlib 차트 → MinIO → 답변 마크다운

```
import requests, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

cdn_url = "http://llmops-cdn-api-service:8080" # 내부망: 코드서빙/코드스페이스에서 실행
hostname = "https://genos.genon.ai" # 사용자가 접속하는 도메인 (스킴 포함)

plt.figure(figsize=(5, 3))
plt.bar(["12m", "24m", "36m"], [4.9, 5.4, 5.9]); plt.title("Installment rate (%)")
plt.savefig("rate.png", bbox_inches="tight"); plt.close()

with open("rate.png", "rb") as f:
    res = requests.post(f"{cdn_url}/minio/upload/temp",
                        files={"file": ("rate.png", f, "image/png")},
                        data={"hostname": hostname}, timeout=30).json()
assert res["code"] == 0, res
presigned_url = res["data"]["presigned_url"]

# 답변에 그대로 실으면 채팅 화면에 이미지가 렌더된다
answer = f"36개월 기준 금리는 5.9% 입니다.\n\n[할부 금리 비교]({presigned_url})"
yield sse("token", answer) # 코드서빙이라면 token 이벤트로
```

상세 설명

왜 MinIO 인가

코드서빙 파드의 로컬 파일은 브라우저가 볼 수 없습니다. MinIO 에 올려 공개 URL 을 받아야 채팅에 이미지가 뜹니다.

만료

presigned URL 은 만료 시간이 있습니다 (URL 의 X-Amz-Expires). 만료 뒤에는 히스토리에 남은 옛 이미지가 깨질 수 있습니다.

기존 객체

이미 MinIO 에 있는 문서 리소스는 /minio/presigned/create 로 URL 만 받습니다 (bucket_name, object_name).

material

GenOSMinIOService.py. upload_and_get_presigned_url() · get_doc_resource_url()

MinIO: Presigned URL 발급

서비스 화면

```
1 import os
2 from Material.GenOSMinIOService import GenOSMinIOService
3
4 hostname = os.getenv("_CLUSTER_HOSTNAME", "")
5 html_path = "/workspace/charts/chart_20260305_170112_115971_fe5b48be.html"
6
7 service = GenOSMinIOService()
8 service.upload_html_and_get_presigned_url(file_path = html_path)
```

'https://genos.genon.ai/minio/temp/0a8799ac-1354-4db5-b9f5-570872c63460...
 .html?response-content-type=text%2Fhtml&response-content-dispos...
 -Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=admin%2F20260305%2...
 %2Faws4_request&X-Amz-Date=20260305T085612Z&X-Amz-Expires=17280...
 -SignedHeaders=host&X-Amz-Signature...
 =9ffbb1145f6f9ff012e4cf7d85f48aa555625943c8967e012540b40cba4373'

결과 요약: 총 7개의 지역단만 해당 조건을 충족하여 결과로 도출되었습니다. 상위 10개를 요청했으나, 조건을 만족하는 데 이터가 7건뿐이므로 전체 7건이 표시되었습니다.

결과 시각화

202501~202505 월초가 40,000원 이상인 계약 기준 건강월초가 가장 높은 지역단 (상위 10개)



상세 설명

Presigned URL 발급

GenOSMinIOService.py의 upload_and_get_presigned_url 메서드를 통해 Local 에 생성된 파일의 임시 배포 URL (Presigned URL)을 받을 수 있음

시각 자료 노출

Chart.html과 같은 차트 html 파일을 입력하여 URL을 받아 올 경우, 채팅 앱 상에서 표시할 수 있음

API 상세

Redis 연결 정보

클러스터 내부 Redis. 코드서빙·Python Step 로직의 세션 상태(멀티턴 히스토리 · HITL 대기 상태) 보관에 씁니다.
Flowise 노드 워크플로우는 히스토리를 자동 처리하지만 직접 짠 로직은 여기에 직접 보관해야 합니다.

- 주소 : [llmops-redis-service](#) : 6379 (내부망. 코드스페이스·코드서빙에서만 접근)
- 인증 : 없음 (db=0). 파이썬 redis 라이브러리 그대로 사용
- 키 설계 : 채팅 세션 ID([x-genos-session-id](#) 헤더 / chatId) 를 키로 쓰는 것이 기본

용도	키 예시	값	Description
멀티턴 히스토리	chat:{session_id}	JSON list [{role, content}]	최근 N 턴만 유지 · ex= 로 만료. /chat 은 매 턴 마지막 질문만 오므로 앱이 직접 보관
HITL 대기 상태	hitl:{interactionId}	JSON {question, step, ...}	action 을 보낸 뒤 humanInput 턴에서 원 질문·단계를 복원
임시 결과 캐시	cache:{hash}	str / JSON	동일 질의 재사용 등 (선택)

Output

- redis.Redis(host="llmops-redis-service", port=6379, db=0). get/set/expire 등 표준 명령

▲ 인메모리 dict 로 상태를 들고 있으면 파드 재시작·복제본 2개 이상(HA)에서 유실. 운영은 Redis 로

Redis 멀티턴 히스토리 예시

세션 ID 를 키로 로드 → 질문 추가 → 답변 생성 → 저장

```
import json, redis
r = redis.Redis(host="llmops-redis-service", port=6379, db=0)
WINDOW = 10 # 최근 N 턴만 유지

def load_history(session_id):
    raw = r.get(f"chat:{session_id}")
    return json.loads(raw) if raw else []

def save_history(session_id, history):
    r.set(f"chat:{session_id}", json.dumps(history[-WINDOW*2:], ensure_ascii=False), ex=60*60*24)

# 코드서빙 /chat 안에서: 세션 ID 는 앱 채팅 경로에서만 헤더로 온다 (외부 인증키 호출은 제거됨)
session_id = request.headers.get("x-genos-session-id", "anon")
history = load_history(session_id)
history.append({"role": "user", "content": question})

messages = [{"role": "system", "content": SYSTEM_PROMPT}] + history
answer = call_llm(messages) # LLM 서빙 호출

history.append({"role": "assistant", "content": answer})
save_history(session_id, history)
```

상세 설명

왜 필요한가

코드서빙 /chat 에는 매 턴 마지막 질문만 옵니다(messages 없음). 이전 대화를 기억하려면 앱이 세션 ID 로 직접 히스토리를 보관해야 합니다.

키는 x-genos-session-id

genportal 이 채팅 세션 ID 를 이 헤더로 보냅니다(외부 인증키 호출에는 오지 않음). HITL 의 interactionId → 상태 매핑도 같은 방식으로 Redis 에 두면 파드가 바뀌어도 이어집니다.

윈도우·만료

토큰 폭주를 막기 위해 최근 N 턴만 남기고, ex= 로 만료를 둡니다.

material

PythonStepRedisHistory.py (Flowise Python Step 판). 코드서빙에서는 위 패턴을 /chat 안에 넣습니다

Redis 연결 및 히스토리 관리

서빗 -화면

```

24
25 redis = genos_import('redis')
26 sid = data.get('socketIOClientId')
27 r = redis.Redis(host='llmops-redis-service', port=6379, db=0)
28 session_id = data.get("chatId", "")
29 question = data.get("question", "")
30 history = []
31
32 # 기존 히스토리 로드
33 if session_id:
34     raw_history = r.get(session_id)
35     if raw_history:
36         history = json.loads(raw_history)
37         print('[History] session_id가 존재하여 히스토리를 불러왔습니다.')
38         print(f'[History] 불러온 히스토리 개수: {len(history)}')
```

NAME↑	TYPE	CLUSTER-IP	EXTERNAL-IP	PORTS	AGE
llmops-redis-service	NodePort			redis:6379-31920	447d

```

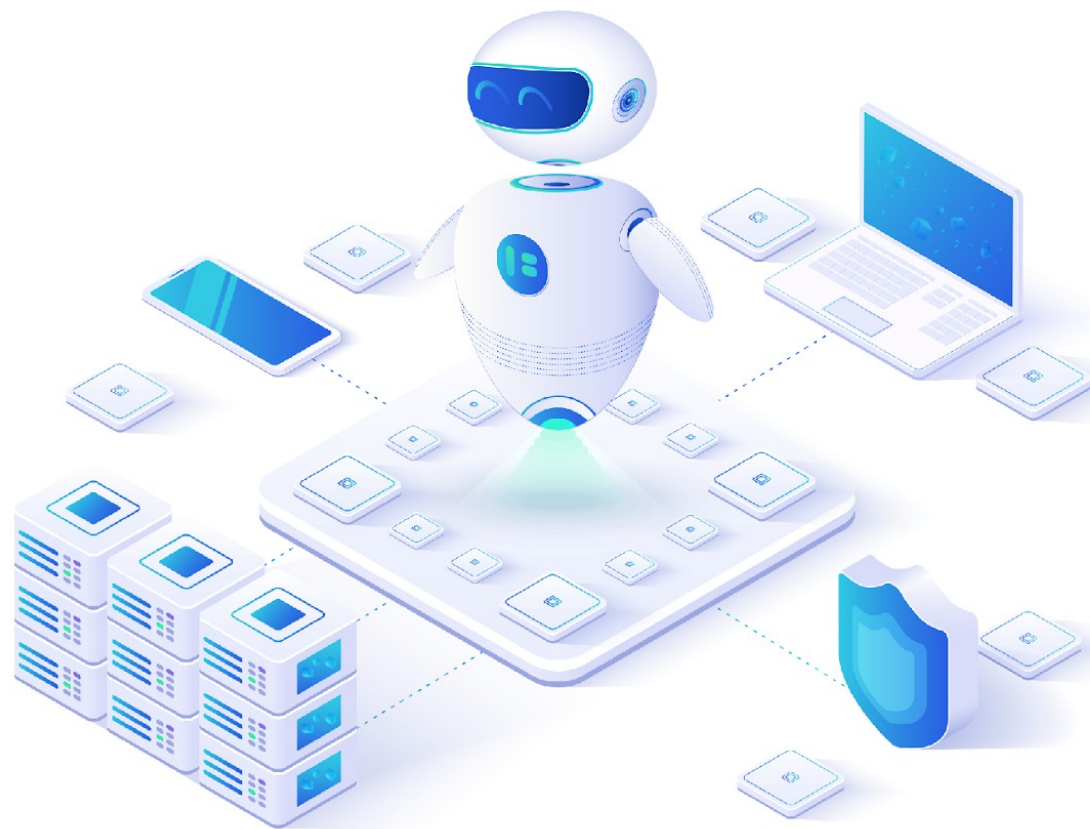
INFO: 2026-03-06 14:51:36 KST|[main.socketio.py:12 - connect()] ws connected: sid='d1HGEOQ0ucQvtuWFAAAB'
INFO: None - "WebSocket /socket.io/?EIO=4&transport=websocket" [accepted]
INFO: connection open
INFO: 2026-03-06 14:51:39 KST|[base.py:88 - process_data()] BYTE processing : data
INFO: - "GET /healthcheck HTTP/1.1" 200 OK
INFO: 2026-03-06 14:51:39 KST|[run_service_v2.py:53 - run_workflow_v2()] Running step 1 of 1 (WS0001)
INFO: 사용자 질문을 history에 추가하였습니다.
model: 640번과 토론이 입력되었습니다.
어시스턴트 응답을 history에 추가하였습니다.
INFO: 2026-03-06 14:51:45 KST|[run_service_v2.py:80 - run_workflow_v2()] Python run completed.
INFO: - "POST /run/v2 HTTP/1.1" 200 OK
INFO: 2026-03-06 14:51:55 KST|[base.py:88 - process_data()] BYTE processing : data
INFO: - "GET /healthcheck HTTP/1.1" 200 OK
INFO: 2026-03-06 14:51:55 KST|[run_service_v2.py:53 - run_workflow_v2()] Running step 1 of 1 (WS0001)
INFO: session_id가 존재하여 히스토리를 불러왔습니다.
불러온 히스토리 개수: 2
사용자 질문을 history에 추가하였습니다.
model: 640번과 토론이 입력되었습니다.
어시스턴트 응답을 history에 추가하였습니다.
INFO: 2026-03-06 14:52:00 KST|[run_service_v2.py:80 - run_workflow_v2()] Python run completed.
INFO: - "POST /run/v2 HTTP/1.1" 200 OK
INFO: 2026-03-06 14:52:05 KST|[main.socketio.py:17 - disconnect()] ws disconnected: sid='d1HGEOQ0ucQvtuWFAAAB'
INFO: connection closed
```

상세 설명

- Redis Service 활용**
 GenOS 클러스터 내에 서비스되는 Redis를 활용
- Redis 연결 및 활용**
 PythonStepRedisHistory.py에 로드 및 저장
 과정이 기술돼 있음
 케이스에 따라 Window 사이즈 조절 및 로깅 가능
- History 구성**
 히스토리 로드 ≡ 질문 저장 ≡ 답변 생성순서 ≡
 답변 저장 순서로 구성 (로직에 따라 변경 가능)

III 채팅 애플리케이션 연동

1. Streaming 답변 생성
2. 추론과정 노출
3. 참조 문서 노출
4. 이미지, 차트 출력
5. 코드서빙 SSE 규약
6. HITL 규약



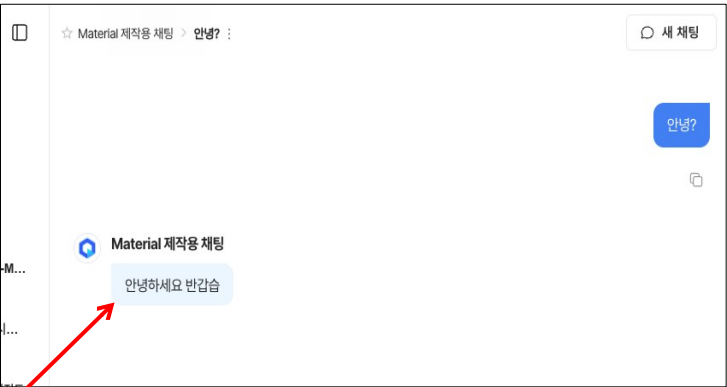
1. Streaming 답변 생성

파이썬 스트리밍 답변 출력 예시

서비스 화면

```

1- async def run(data):
2-     import asyncio
3-     import inspect
4-     import json
5-     try:
6-         from main_socketio import sio_server
7-     except ImportError:
8-         sio_server = None
9-     sid = data.get('socketIOClientId')
10-
11-
12-     async def emit_event(event_name, payload):
13-         if sio_server and sid:
14-             await sio_server.emit(event_name, payload, room=sid)
15-         return {"event": event_name, "data": payload}
16-
17-
18-     async def return_answer(iterable):
19-         answer = ""
20-         if inspect.isasyncgen(iterable):
21-             async for token in iterable:
22-                 answer += token
23-                 await asyncio.sleep(0.5)
24-                 yield await emit_event('token', token)
25-         else:
26-             for token in iterable:
27-                 answer += token
28-                 await asyncio.sleep(0.5)
29-                 yield await emit_event('token', token)
30-
31-         yield {"event": "result", "data": {"text": answer}}
```



상세 설명

Flowise 에서의 Streaming

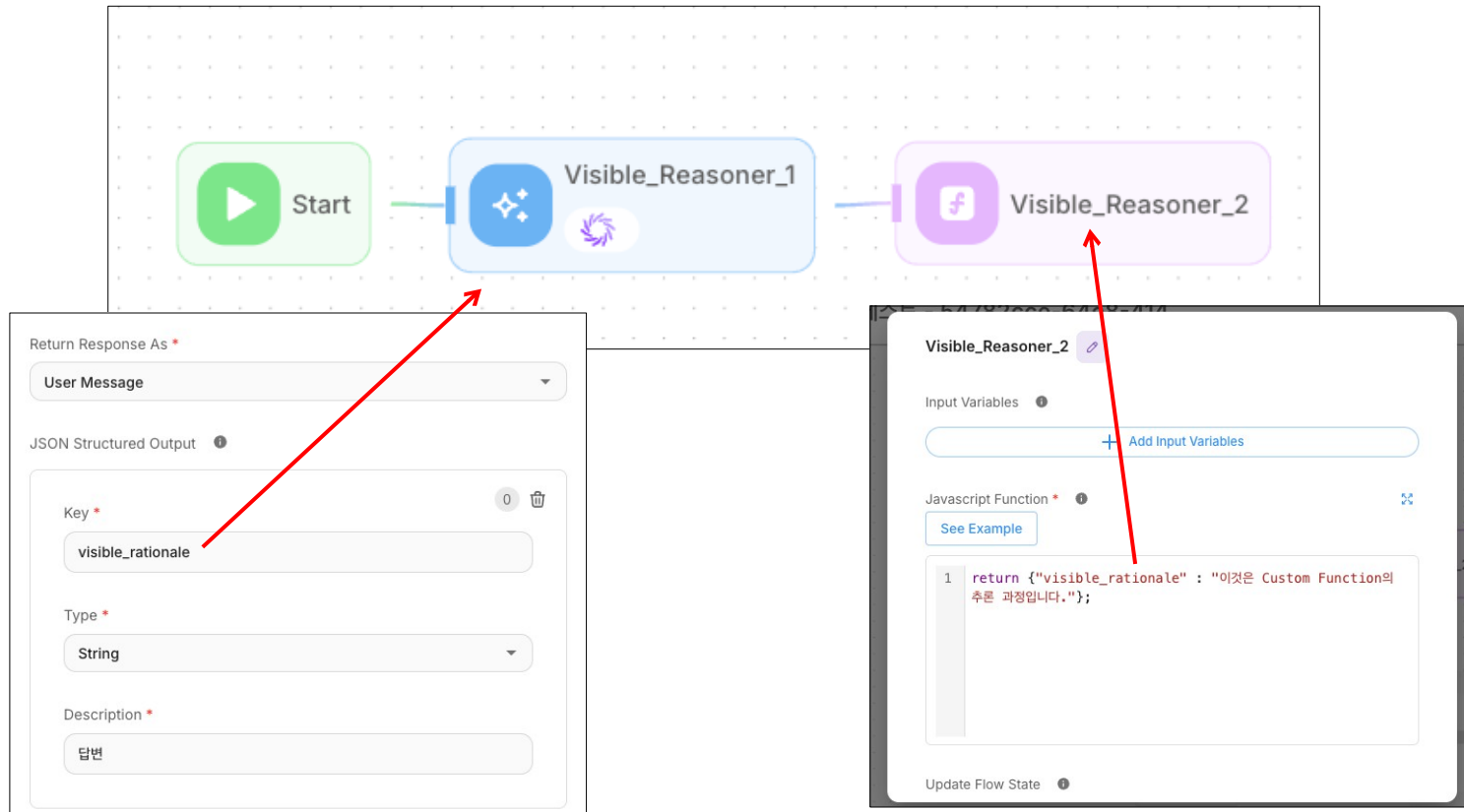
Flowise 노드로 구성된 워크플로우는 자동으로 스트리밍됩니다.

Python Step 에서의 Streaming

워크플로우 컨테이너의 sio_server.emit('token', 조각) 으로 토큰을 흘리고, 마지막에 {"event": "result", "data": {"text": 전체}} 를 yield 합니다. 컨테이너(run_util_v2)가 token · result · sourceDocuments · action · agentFlowExecutedData 이벤트를 처리합니다. (PythonStepStreaming.py)

코드서빙(FastAPI)은 Socket IO 가 아니라 SSE 로 data: {"event": "token", "data": 조각} 프레임을 보냅니다.

Flowise에서의 추론 과정 노출 예시



상세 설명

- Flowise에서의 Reasoning**
Flowise Step에서는 노드명이 "Visible_Reasoner"인 LLM 혹은 Custom Function 노드의 Output이 {visible_rationale" : "추론과정"} 형태로 Return되면 추론 과정으로 노출됨

추론 과정 예시



서비스 화면

```
question = data.get('question', "")
retrieved_documents = vdb.hybrid_search(query = question)
system_prompt = f"검색된 문서를 바탕으로 질문에 대답하세요."
검색된 문서 : {retrieved_documents}
answer = llm.call(system_prompt = system_prompt, question = question)
data['text'] = answer
sourceDocuments = [
    {
        "pageContent": i['text'],
        "metadata": {"file_path": i['file_path'], "file_name": i['file_name'],
    } for i in retrieved_documents
]
data['sourceDocuments'] = sourceDocuments
return data
```

도로교통법이 뭐야

Material 제작용 채팅 - 5.3s

도로교통법은 도로에서 발생하는 교통상의 모든 위험과 장애를 방지하고 제거하여, 안전하고 원활한 교통을 확보함을 목적으로 하는 법률입니다(제1조).

주요 내용은 다음과 같습니다:

1. 도로의 정의 및 규칙: 고속도로, 자동차전용도로, 차도, 보도 등 도로의 개념을 정의하고 보행자와 차량의 통행 방법을 규정합니다.
2. 운전자 준수 사항: 음주운전, 난폭운전 금지 및 서행, 앞차르기, 일시정지 등 운행 규칙을 다룹니다.
3. 보호구역 관리: 어린이·노인 보호구역의 지정 및 관리 기준을 포함합니다.
4. 면허 및 교육: 운전면허의 취득과 취소 기준, 자동차운전학원 운영, 교통안전교육에 관한 사항을 규정합니다.
5. 신기술 및 특수차량: 자율주행자동차 운전자의 의무, 어린이통학버스 신고 및 관리, 개인형 이동장치 등에 대한 규정을 담고 있습니다.
6. 행정 처분: 위반 시 부과되는 과태료, 범칙행위 처리 절차 및 벌칙 내용을 포함합니다.

이 법은 경찰청 소관으로, 도로 위 교통 안전 및 질서 확립을 위한 가장 핵심적인 법적 근거가 됩니다.

출처

1 도로교통법 시행규칙(행정안전부령)(제00610호)(20260224).pdf

HEADER: 도로교통법 시행규칙, [별지 제109호서식] 식제, [별지 제110호서식] 식제, [별지 제111호서식] ○○자동차운전(전문)학원 직인등록 대장 도로교통법 시행규칙, [별지 제109호서식] 식제, [별지 제109호서식] 식제 <2011.4.30> 도로교통법 시행규칙, [별지 제110호서식] 식제, [별지 제110호서식] 식제 <2011.4.30> 도로교통법 시행규칙, [별지 제111호서식] ○○자동차운전(전문)학원 직인등록대장, [별지 제111호서식] 식제

2 도로교통법(법률)(제21016호)(20260101).pdf

HEADER: 도로교통법, 제1장 총칙, 제1조(목적) 도로교통법, 도로교통법 도로교통법, 제1장 총칙, 제1장 총칙 도로교통법, 제1장 총칙, 제1조(목적) 도로교통법, 제1조(목적) 이 법은 도로에서 일어나는 교통상의 모든 위험과 장애를 방지하고 제거하여 안전하고 원활한 교통을 확보 함을 목적으로 한다.

3 도로교통법(법률)(제21016호)(20260101).pdf

HEADER: 도로교통법, 제12장 보칙, 제139조(수수료) 도로교통법, 제12장 보칙, 제139조(수수료), 제139조(수수료) ○ 다음 각 호의 어느 하나에 해당하는 사람은 행정안전부령으로 정하는 바에 따라 수수료를 내야 한다. 다만, 경찰청장 또는 시·도경찰청장이 제147조에 따라 입무를 대행하게 한 경우에는 그 입무를 대행하는 한국 도로교통공단이 경찰청장의 승인을 받아 결정·공고하는 수수료를 한국도로교통공단에 내야 한다.

4 도로교통법(법률)(제21016호)(20260101).pdf

HEADER: 도로교통법, 제1장 총칙, 제2조(정의) 도로교통법, 제1장 총칙, 제2조(정의), 제2조(정의) 이 법에서 사용하는 용어의 뜻은 다음과 같다. <개정 2012. 3. 21, 2013. 3. 23, 2014. 1. 28, 2014. 11. 19, 2017. 3. 21, 2017. 7. 26, 2017. 10. 24, 2018. 3. 27, 2020. 5. 26, 2020. 6. 9, 2020. 12. 22, 2021. 10. 19, 2022. 1. 11, 2023. 4. 18, 2023. 10. 24, 2025. 12. 30.> 1. "도로"란 다음 각 목에 해당하는 곳을 ...

상세 설명

PDF 뷰어 활성화

sourceDocuments 이벤트(또는 비스트리밍 data.-sourceDocuments)에 문서 정보가 있으면 채팅 화면 하단 「출처」카드와 PDF 뷰어가 열립니다.

sourceDocuments 규약

```
{["pageContent": 청크 본문, "metadata": {"file_name": "file_name", "file_path": "file_path", "i_page": "i_page", "e_page": "e_page", "doc_id": "doc_id", "security_level": "security_level", "chunk_bboxes": "..."}]}
```

metadata 는 text 를 뺀 청크 프로퍼티 전부를 넘긴다

출처 카드는 file_name · i_page 만 보이지만, 뷰어 하이라이트(chunk_bboxes) · 썸네일(media_files) · 보안등급 필터(security_level) · 휴지통 제외(doc_id)가 metadata 를 읽습니다. 화이트리스트로 고르면 기능이 조용히 빠집니다. (GenOSVectorDB.to-source_documents · PythonStepRAGWorkflow.py)

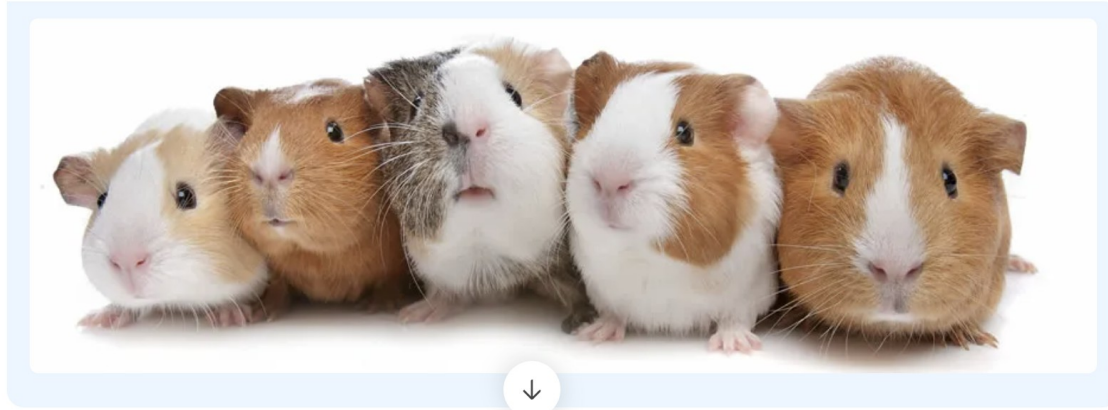
서비스 화면

7/7 <> 답변 내 이미지 노출

```
1 async def run(data: dict) -> dict:
2     # If only Python steps are used, the "text" field is required
3     data['text'] = "![Guinea Pig](https://i.namu.wiki/i/BtyFoNFHn1GqmP2)"
4     return data
5
```

안녕 ?

Material 제작용 채팅 • 1.3s



상세 설명

- 채팅 어플리케이션 내에서의 이미지 노출
LLM의 답변이 Markdown 문법에 맞도록 링크를 생성하면 답변창에서 이미지 노출 가능
문법 예시 `![이미지 제목](image_url)`

코드서빙 SSE 이벤트 규약

API 상세

코드서빙 → 채팅앱 SSE 프레임

코드서빙 워크플로우가 앱 채팅에 연결됐을 때 보내야 하는 형식입니다. genportal-api 가 프레임을 받아 프론트로 전달합니다.

- 프레임 : data: {"event": ..., "data": ...}\n\n . data: 뒤 공백 필수, 한 줄 JSON, 빈 줄로 종료. 그 외 줄은 전부 버려짐
- 헤더 : Content-Type: text/event-stream · X-Accel-Buffering: no
- 비스트리밍(stream 없음) : SSE 대신 JSON 봉투 {"code": 0, "errMsg": "success", "data": {"text", "sourceDocuments"}}

event	data	용도	Description
token	str	본문	답변 조각. 본문을 심는 유일한 이벤트. <think>...</think> 로 감싸면 추론 영역으로 접힘(단, GenOS 앱채팅 설정에 토큰을 등록하여야 함.)
sourceDocuments	list[dict]	출처	[{pageContent, metadata}]. 교체 방식(마지막 값 채택). 빈 배열은 무시되므로 보내지 않음
action	dict	HITL	선택·승인 UI. 다음 슬라이드
error	str	실패	"[코드] 메시지" 형식 권장. 프론트가 코드별 안내 문구로 바꿈
end	null	종료	genportal 이 자체 emit 하므로 실제로는 무시되지만 보내는 것이 규약

HITL(action) 규약: hitl-ui/v1

API 상세

action 이벤트: 사람의 선택·승인 받기

선택지나 승인 버튼을 띄우고 스트림을 끝냅니다. 사용자가 응답하면 다음 턴에 humanInput 으로 돌아옵니다.

- 요청 : data: {"interactionId": uuid, "elements": [...]} . **elements** 는 배열(아니면 무반응), 각 **element** 에 **interactionId** 필수
- element : {"type", "interactionId", "component": {"type", "title"(질문), "options" [{value, label, desc?}]}} . 직접 입력은 FE 가 자동 추가
- **승인/거절(confirm)은 element 두 개** : type approve-button + reject-button, component.type 은 confirm, title 은 approve 쪽에
- 응답 : 다음 요청 body {"question": "", "humanInput": {type, interactionId, action: submit|cancel, values: {selected[], customInput}}} .
원 질문이 안 오므로 interactionId 로 상태 보관

component	element	화면	humanInput
single-select	1개 (type: single-select)	라디오 + 제출 (+ 직접 입력)	action: submit, values.selected: [value], customInput
multi-select	1개 (type: multi-select)	체크박스 + 제출	action: submit, values.selected: [value, ...]
confirm	2개 (approve-button, reject-button)	답변 진행 / 답변 거절	submit(승인) / cancel(거절, +feedback)

IV 에이전트 개발



에이전트 개발 방향성 3가지

1

Flowise 워크플로우 + GenOS 앱 채팅

빠른 PoC

- Flowise 화면에서 LLM·벡터DB·도구 노드를 연결해 워크플로우를 설계하고, 앱 채팅에 연결해 바로 사용
- 멀티턴 히스토리·스트리밍·출처 뷰어·가드레일·이용 로그를 GenOS가 자동 처리
- 대상: 현업 담당자, 비개발자, 아이디어를 빠르게 검증하려는 팀

2

코드서빙 백엔드 + 자체 Front-End

FastAPI 등 코드 · 자체 UI 보유 개발팀

- 채팅 로직을 FastAPI 등으로 구현해 코드서빙으로 배포하고, 자체 Front-End가 게이트웨이 API로 호출
- 인증·크레딧·가드레일·이용 로그·배포 승인 등 GenOS가, 멀티턴·스트리밍·에러 처리는 코드에서 직접 구현
- 대상: 자체 UI가 있고 백엔드 운영(배포·인증·과금·로그)만 GenOS에 맡기려는 개발팀

3

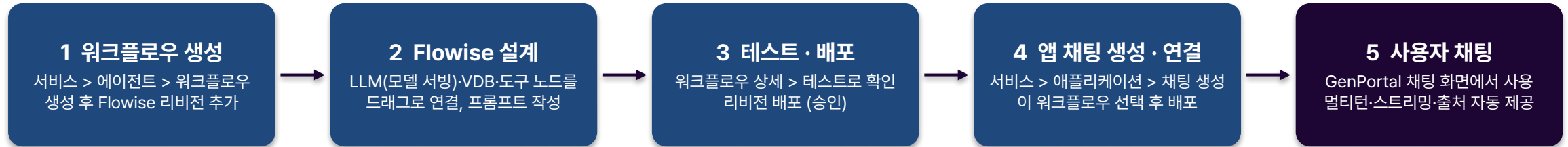
GenOS 리소스 API 직접 호출

기존 시스템에 AI 기능 삽입

- LLM·임베딩·워크플로우·MCP·A2A API를 자체 애플리케이션에서 직접 호출
- OpenAI 호환 규격이라 기존 코드의 base_url과 인증키만 바꿔 연결 가능
- 대상: 이미 운영 중인 시스템(포털·업무 시스템)에 AI 기능만 추가하려는 경우

① Flowise 워크플로우 → 앱 채팅 연결 흐름

Flowise로 워크플로우를 만들고 앱 채팅에 연결하면, 사용자는 GenPortal 채팅 화면에서 바로 사용합니다. 코드 작성이 필요 없습니다.



이 흐름에서 GenOS가 자동으로 처리하는 것

멀티턴 히스토리

앱 채팅이 세션별 대화를 저장하고 매 턴 워크플로우에 messages로 전달하기에 UI 내에서의 설정만 필요합니다.

스트리밍 출력

토큰 단위 응답을 채팅 화면 규약으로 변환해 실시간 표시

참조 문서 뷰어

sourceDocuments 규약만 지키면 PDF 뷰어와 출처 카드 자동

가드레일

워크플로우 플러그인 탭에서 입력·출력 가드레일 선택 후 재배포

인증 · 크레딧 · 이용 로그

SSO 사용자 인증, 턴 단위 크레딧 차감, 이용 로그·탐지 로그 기록

중지

사용자 중지 버튼

① 제작자가 설정하는 것 vs GenOS가 처리하는 것

워크플로우 제작자가 Flowise에서 설정

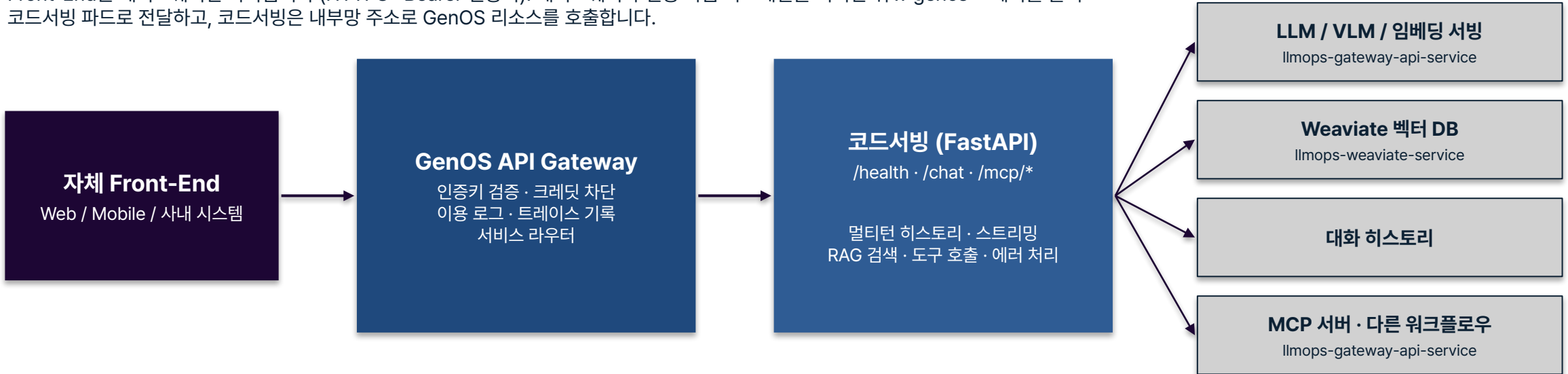
- **LLM 노드 · 모델 서빙 선택**
배포된 LLM 서빙 지정, 시스템 프롬프트 설정
- **지식 연결 · Document Store**
벡터 DB에 문서 적재 후 Flowise Document Store에서 인덱스 지정
- **도구 · MCP 노드**
배포된 MCP 서버를 도구로 연결, 에이전트 노드가 필요할 때 호출
- **가드레일 플러그인**
워크플로우 상세 > 플러그인 탭에서 입력/출력 가드레일 활성화 후 재배포
- **앱 채팅 연결**
채팅 서비스 생성 시 이 워크플로우를 선택. 이후 변경은 리비전 재배포로 반영

GenOS 앱 채팅이 자동으로 처리

- **대화 히스토리**
세션(chatId)별 질문·답변을 저장하고 다음 턴에 messages 배열로 전달
- **스트리밍 변환**
Flowise 토큰 이벤트를 채팅 화면 규약 {"event": "token"}으로 변환해 실시간 표시
- **참조 문서 · 보안 등급**
sourceDocuments를 PDF 뷰어로 표시, 사용자 보안 등급보다 높은 문서는 자동 제외
- **첨부 파일 처리**
채팅창 첨부 파일을 임시 VDB로 적재하고 질문에 컨텍스트로 주입
- **인증 · 과금 · 로그**
SSO 사용자 식별, 턴 단위 크레딧 차감, 이용 로그·가드레일 탐지 로그 기록
- **중지 · 비상 차단**
사용자 중지 버튼, 관리자 킬스위치 발동 시 진행 중 응답 종료 및 안내 문구 표시

② 코드서빙 백엔드 아키텍처

Front-End는 게이트웨이만 바라봅니다 (HTTPS · Bearer 인증키). 게이트웨이가 인증·과금·가드레일을 처리한 뒤 x-genos-* 헤더를 붙여 코드서빙 파드로 전달하고, 코드서빙은 내부망 주소로 GenOS 리소스를 호출합니다.



GenOS가 맡는 것

- 인증키 발급·검증, 허용 IP 제한
- 크레딧 차감, 월 예산 초과 시 차단
- 이용 로그
- 리비전 배포·승인

코드서빙에서 구현하는 것

- /health, /chat 엔드포인트와 같은 응답 규약
- 세션 ID 기반 멀티턴 히스토리
- SSE 스트리밍
- LLM·VDB·MCP 내부 호출, 프롬프트 설계
- 예외 처리 · 애플리케이션 로깅

② 코드서빙 최소 구현: 필수 엔드포인트와 응답 규약

main.py. 최소 골격 (material/GenOSCodeServing_main.py 축약)

```
import json, os, httpx
from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse
app = FastAPI()
LLM_URL = f"{os.getenv('GENOS_URL')}/api/gateway/rep/serving/{os.getenv('LLM_SERVING_ID')}/v1/chat/completions"
HEADERS = {"Authorization": f"Bearer {os.getenv('LLM_BEARER_TOKEN')}" } # 설정은 env 로 주입

def sse(event, data):
    # data: 뒤 공백 필수, 한 줄 JSON, 빈 줄로 종료
    return "data: " + json.dumps({"event": event, "data": data}, ensure_ascii=False) + "\n\n"

@app.get("/health")
# startupProbe 가 호출. 없으면 배포 실패
async def health(): return {"status": "ok"}

@app.post("/chat")
# 컨테이너 서비스 탭 Chat Path 와 동일
async def chat(request: Request):
    body = await request.json(); q = body.get("question", "")
    if q == "__verify__":
        # 검증 버튼 → LLM 없이 즉답
        return {"code": 0, "errMsg": "success", "data": {"text": "verified"}}
    if not body.get("stream"):
        # 어드민 테스트 등 비스트리밍 → JSON 봉투
        return {"code": 0, "errMsg": "success", "data": {"text": await answer_text(q)}}
    async def gen():
        # 채팅앱 → SSE 이벤트
        async for token in llm_stream(q): yield sse("token", token)
    yield sse("end", None)
    return StreamingResponse(gen(), media_type="text/event-stream",
        headers={"X-Accel-Buffering": "no"})
```

상세 설명

GET /health 필수

쿠버네티스 startupProbe 가 호출. 200 이 없으면 배포 실패.

레포 루트 main.py + app

entrypoint 가 uvicorn main:app --port 8080 으로 자동 기동.
시작 커맨드·Dockerfile 불필요.

두 응답 형식

stream 없음 → {"code":0,"data":{"text"}} JSON. stream=true
→ SSE data:{"event":token|sourceDocuments|action|error|
end}. 실패도 HTTP 200 + code 1.

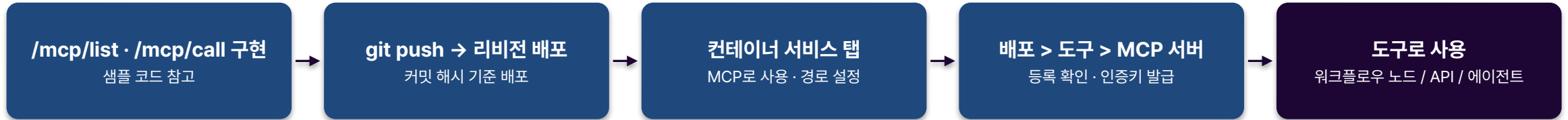
__verify__

검증 버튼·부하테스트가 보내는 값. LLM 을 부르지 않고 code 0 즉답.

설정엔 env 로

리비전 > 환경 설정. 비밀값은 암호화 행, 나머지는 시작 커맨드에
VAR=값 으로. ▲ WEAVIATE_PORT 처럼 <서비스명>_PORT
이름은 K8s 가 tcp://... 로 덮어씀.

② MCP 서버 배포: 코드서빙을 도구로 노출



POST /mcp/list 응답. 도구 목록 (봉투 필수)

```
{
  "code": 0, "errMsg": "success",
  "data": {
    "tools": [
      {
        "name": "get_loan_rate",
        "description": "상품 코드로 현재 할부 금리를 조회한다",
        "inputSchema": {
          "type": "object",
          "properties": {
            "product_code": {
              "type": "string"
            }
          },
          "required": ["product_code"],
          "annotations": {
            "readOnlyHint": true
          }
        }
      }
    ]
  }
}
```

▲ code 가 0 이 아니거나 봉투가 없으면 GenOS 는
도구 목록을 빈 배열로 처리한다 (에러 없이 조용히)

POST /mcp/call 요청 → 응답

```
# 요청 (게이트웨이가 JSON-RPC tools/call 을 이 형식으로 변환)
{
  "name": "get_loan_rate",
  "arguments": {
    "product_code": "HC-AUTO-36"
  }
}

# 응답: MCP 표준 content 배열을 봉투에 담는다
{
  "code": 0, "errMsg": "success",
  "data": {
    "content": [
      {
        "type": "text",
        "text": "HC-AUTO-36 현재 금리: 연 5.9%"
      }
    ]
  }
}
```

오류: code != 0 이면 GenOS 가 errMsg 를 그대로 사용자에게 올린다
{ "code": 1, "errMsg": "상품 코드를 찾을 수 없습니다", "data": null }

• 도구 설명이 곧 프롬프트

description과 inputSchema를 LLM이 읽고 호출 여부와 인자를 결정하므로 구체적으로 작성

• /chat과 /mcp/*를 한 코드서빙에

컨테이너 서비스 탭에서 워크플로우 · MCP 토글을 각각 켜면 하나의 파드가 채팅과 도구를 함께 제공

• 외부에서 도구 호출

{genos_url}/api/gateway/mcp/{mcp_server_id}/mcp 에 JSON-RPC tools/call. 인증키는 MCP 서버 상세에서 발급



감사합니다